Audit de configuration et durcissement d'un serveur web Apache

---

Plan

1. **Phase 1** : Mise en place de l'environnement de travail
2. **Phase 2** : Audit de sécurité initial
3. **Phase 3** : Analyse des résultats et évaluation des risques
4. **Phase 4** : Durcissement du système d'exploitation
5. **Phase 5** : Durcissement du serveur web Apache
6. **Phase 6** : Sécurisation des communications (HTTPS / TLS)
7. **Phase 7** : Audit post-durcissement et vérification finale

---

Phase 1 : Mise en place de l'environnement de travail (Web Server Setup)

---

1. Introduction de la phase

La première phase du projet **Audit et durcissement d'un serveur web** consiste à mettre en place une **infrastructure de base fonctionnelle**, servant de fondation à l'ensemble des étapes suivantes. À ce stade, aucun audit de sécurité n'est encore réalisé. L'objectif principal est de disposer d'un **serveur web opérationnel, stable et contrôlé**, condition indispensable à toute démarche d'audit ou de durcissement.

En effet, un audit de sécurité n'a de valeur que si le service ciblé fonctionne correctement. Cette phase représente donc la **situation de référence (baseline)** qui sera utilisée pour comparer l'état du serveur avant et après l'application des mesures de sécurité.

Le travail est mené selon une **méthodologie académique de niveau intermédiaire**, claire, reproductible et défendable aussi bien dans un rapport écrit que lors d'une présentation orale.

---

2. Scénario et choix techniques

Le scénario retenu pour ce projet est le suivant :

- Utilisation de **Kali Linux** exécuté sur **VMware**
- Kali Linux joue à la fois le rôle de :
  - Serveur web
  - Environnement de test
- Installation du serveur web **Apache**
- Déploiement :
  - D'une page web simple pour vérifier le bon fonctionnement du serveur
  - D'une application web volontairement vulnérable **DVWA (Damn Vulnerable Web Application)** destinée aux phases d'audit ultérieures

Ce scénario est **pleinement acceptable dans un cadre académique**, car il permet de démontrer à la fois la mise en place technique du service et l'analyse de ses faiblesses de sécurité dans un environnement maîtrisé.

---

3. Préparation et mise à jour du système

Avant toute installation logicielle, le système doit être mis à jour afin d'éviter les problèmes de dépendances et de garantir la compatibilité des paquets.

```shell
sudo apt update
sudo apt upgrade -y
```

Objectif :
Assurer que le système d'exploitation et les paquets installés sont à jour et stables.

---

4. Installation et activation du serveur web Apache

Le serveur web **Apache** est installé, car il s'agit du serveur HTTP le plus couramment utilisé dans les projets académiques et professionnels.

```shell
sudo apt install apache2 -y
```

Activation du service Apache :

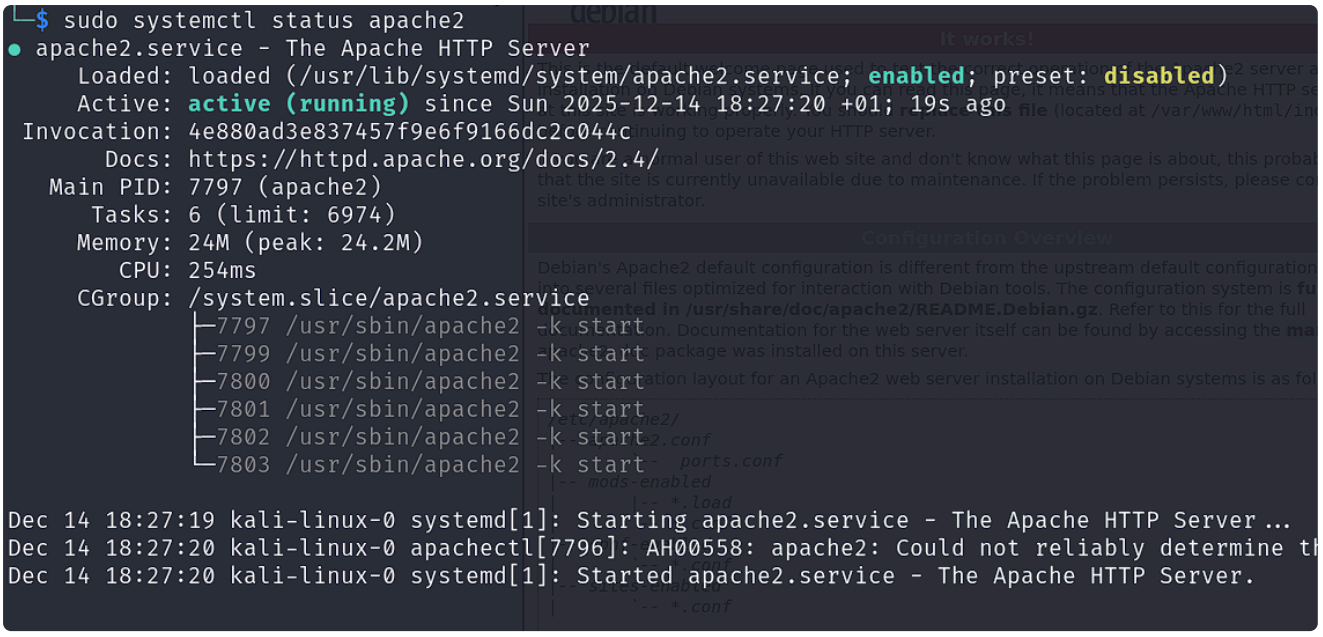
```shell
sudo systemctl start apache2
sudo systemctl enable apache2
```

Vérification de l'état du service :

```shell
sudo systemctl status apache2
```

L'état attendu est : **active (running)**.

- Terminal montrant la commande `systemctl status apache2` && La ligne indiquant que le service est actif et en cours d'exécution doit être visible



---

5. Vérification du fonctionnement du serveur web

Une fois Apache lancé, son bon fonctionnement est vérifié via un navigateur web.

Accès depuis la machine virtuelle :

```
http://localhost
```

Ou depuis la machine hôte en utilisant l'adresse IP de la VM.

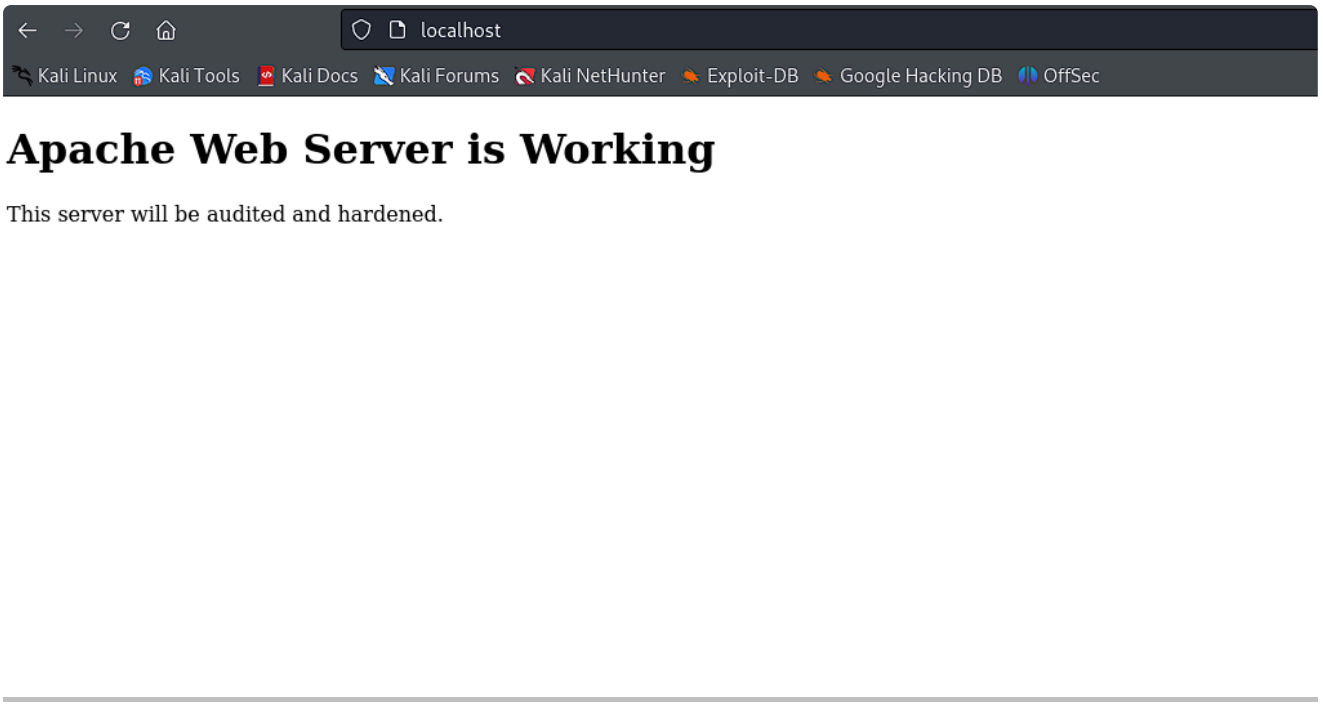
Obtention de l'adresse IP :

```shell
ip a
```

L'apparition de la page **Apache2 Debian Default Page** confirme que :

- Le serveur web est fonctionnel
- Apache est correctement installé
- La configuration réseau est opérationnelle

- Navigateur affichant la page par défaut Apache && L'URL visible dans la barre d'adresse



---

6. Compréhension de l'arborescence web

Le répertoire par défaut des fichiers web Apache est :

```
/var/www/html
```

Affichage du contenu :

```shell
ls /var/www/html
```

Ce répertoire contient généralement le fichier `index.html` servant de page d'accueil.

- Terminal affichant le contenu du répertoire `/var/www/html`



---

7. Création d'une page web personnalisée

Afin de démontrer la maîtrise de l'environnement et enrichir le rapport, une page HTML simple est créée manuellement.

```shell
sudo nano /var/www/html/index.html
```

Contenu de la page :

```HTML
<!DOCTYPE html>
<html>
<head>
    <title>Test Web Server</title>
</head>
<body>
    <h1>Apache Web Server is Working</h1>
    <p>This server will be audited and hardened.</p>
</body>
</html>
```

Cette étape permet de prouver que le serveur héberge un contenu personnalisé et non uniquement une page par défaut.

- **Navigateur affichant la nouvelle page HTML && Le texte personnalisé doit être clairement visible**



---

8. Installation de PHP

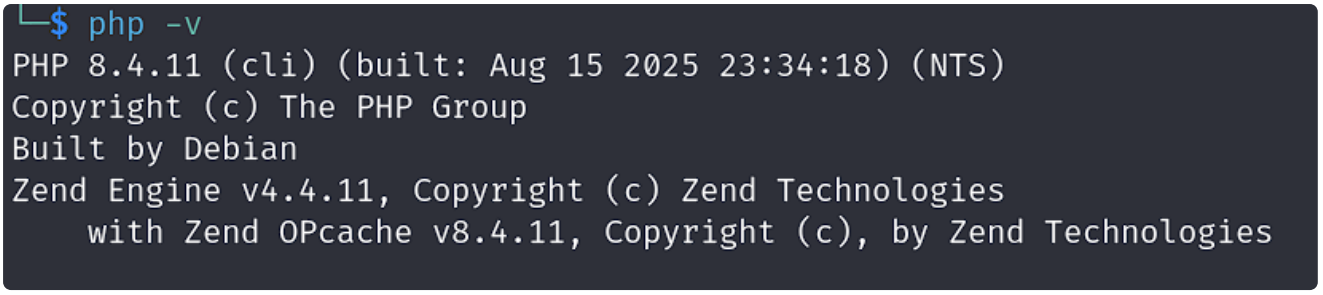
L'application DVWA nécessite PHP pour fonctionner.

```SHELL
sudo apt install php php-mysqli php-gd php-xml php-cli php-zip libapache2-mod-php -y
```

Vérification de la version de PHP :

```SHELL
php -v
```

- Terminal affichant la version installée de PHP



---

9. Installation de la base de données MariaDB

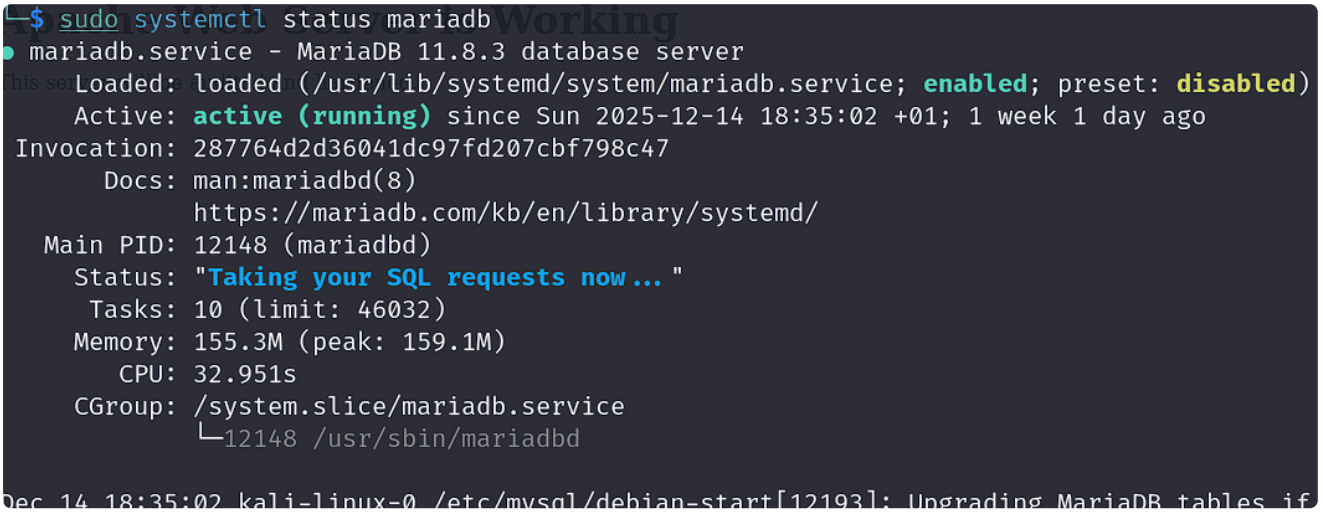
Installation de MariaDB :

```shell
sudo apt install mariadb-server mariadb-client -y
```

Activation du service :

```shell
sudo systemctl start mariadb
sudo systemctl enable mariadb
sudo systemctl status mariadb
```

- Terminal montrant que le service MariaDB est actif



---

10. Sécurisation minimale de MariaDB

Une configuration de sécurité de base est appliquée :

```shell
sudo mysql_secure_installation
```

Choix appliqués :

- Définition d'un mot de passe root
- Suppression des utilisateurs anonymes
- Désactivation de l'accès distant pour root
- Suppression de la base de test
- Rechargement des privilèges

---

11. Installation de DVWA

Téléchargement de DVWA dans le répertoire web :

```shell
cd /var/www/html
sudo git clone https://github.com/digininja/DVWA.git
```

Attribution des permissions :

```shell
sudo chown -R www-data:www-data DVWA
sudo chmod -R 755 DVWA
```

- Terminal montrant le clonage du dépôt DVWA && Permissions appliquées



---

12. Configuration de la base de données DVWA

Connexion à MariaDB :

```shell
sudo mysql -u root -p
```

Création de la base et de l'utilisateur :

```sql
CREATE DATABASE dvwa;
CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'dvwa';
GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
FLUSH PRIVILEGES;
EXIT;
```

- Terminal montrant la création de la base de données DVWA

```
┌──(root㉿kali-linux-0)-[/var/www/html]
└─# sudo mysql -u root -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 13
Server version: 11.8.3-MariaDB-1+b1 from Debian -- Please help get to 10k stars at https://github.com/MariaDB/Server

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> CREATE DATABASE dvwa;
ERROR 1007 (HY000): Can't create database 'dvwa'; database exists
MariaDB [(none)]> CREATE USER 'dvwa'@'localhost' IDENTIFIED BY 'dvwa';
Query OK, 0 rows affected (0.046 sec)

MariaDB [(none)]> GRANT ALL PRIVILEGES ON dvwa.* TO 'dvwa'@'localhost';
Query OK, 0 rows affected (0.004 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> EXIT;
Bye
```

---

13. Configuration de DVWA

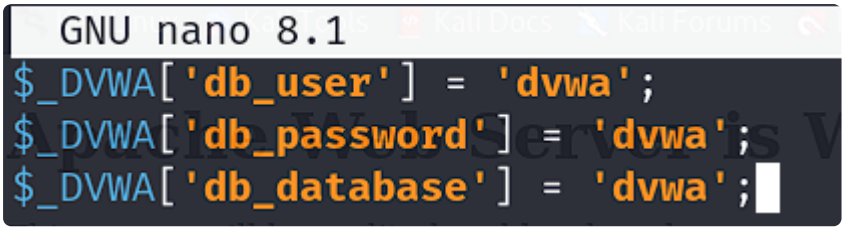
Copie et modification du fichier de configuration :

```shell
cd /var/www/html/DVWA/config
sudo cp config.inc.php.dist config.inc.php
sudo nano config.inc.php
```

Paramètres modifiés :

```php
$_DVWA['db_user'] = 'dvwa';
$_DVWA['db_password'] = 'dvwa';
$_DVWA['db_database'] = 'dvwa';
```

- Éditeur affichant les paramètres DVWA modifiés



---

14. Ajustement des paramètres PHP

Modification du fichier `php.ini` :

```shell
sudo nano /etc/php/*/apache2/php.ini
```

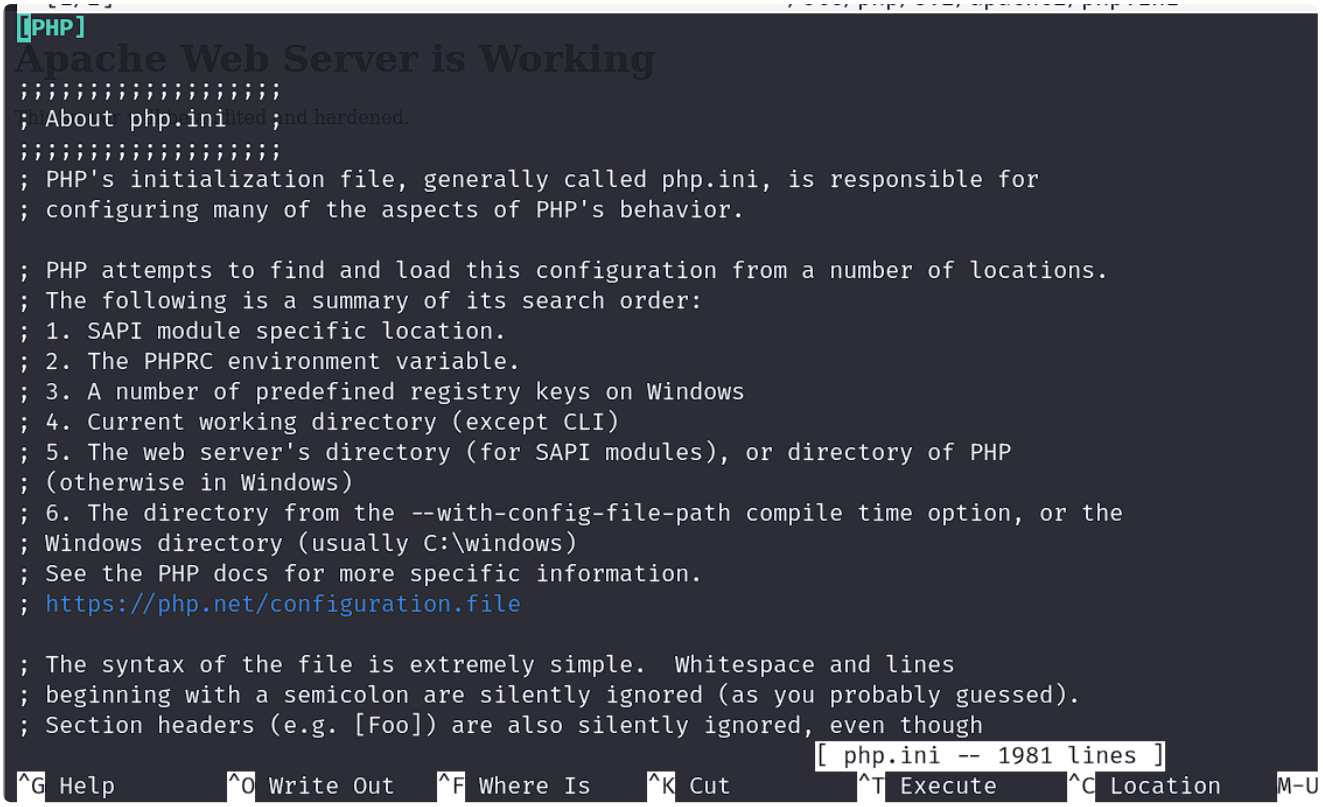
Paramètres activés :

```ini
allow_url_fopen = On
allow_url_include = On
display_errors = On
```

Redémarrage d'Apache :

```shell
sudo systemctl restart apache2
```

- **Fichier `php.ini` montrant les paramètres activés**



---

15. Lancement de DVWA

Accès à l'application :

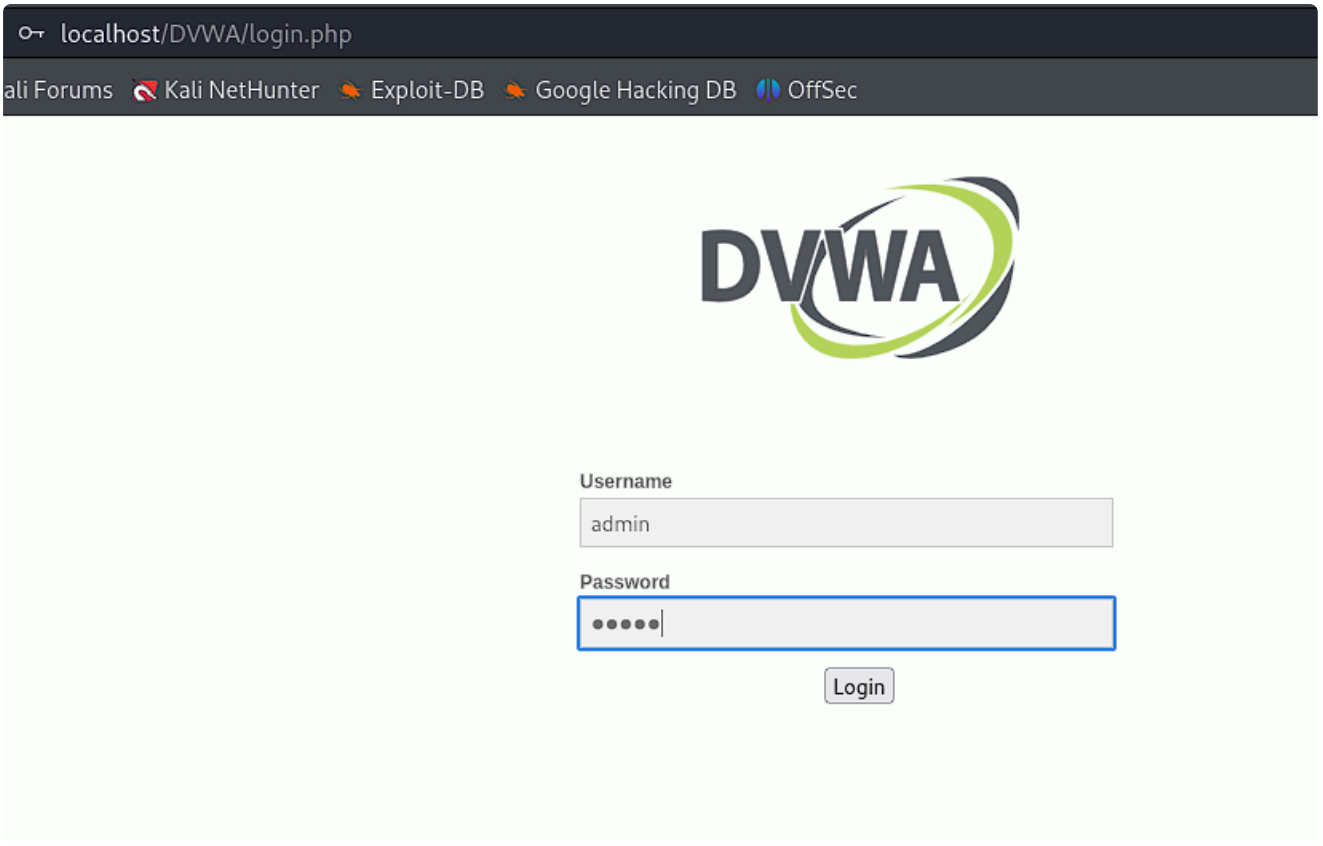
```
http://localhost/DVWA
```

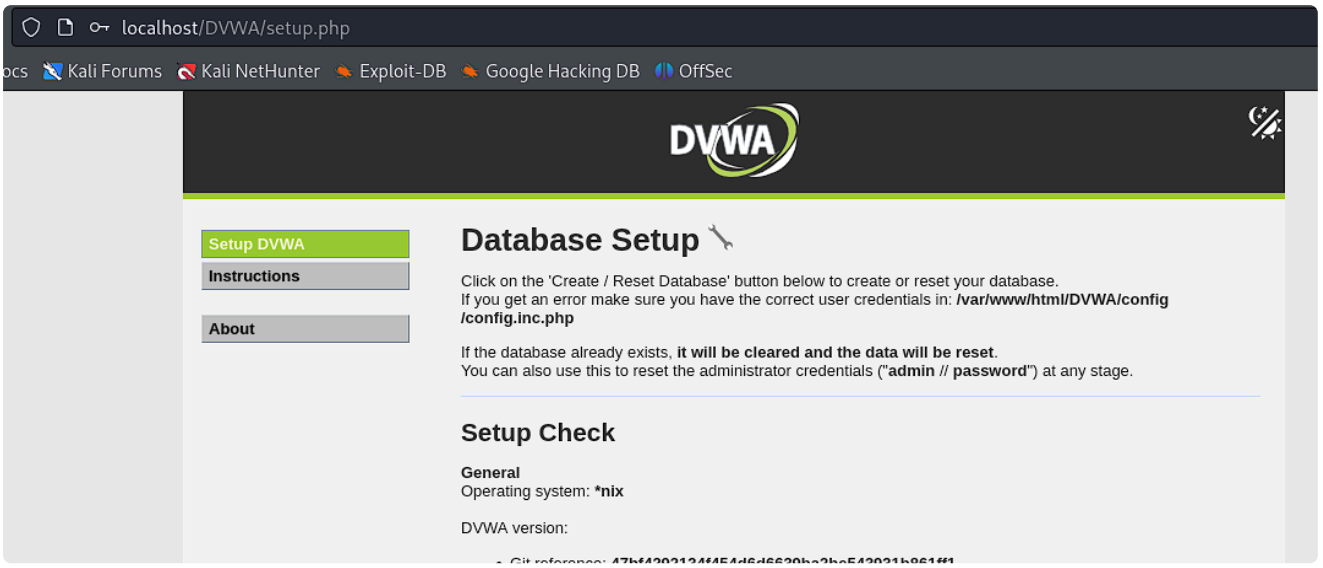
Identifiants par défaut :

- Username : `admin`
- Password : `password`

Initialisation de la base de données via le bouton **Create / Reset Database**.

- Interface DVWA après initialisation réussie





16. Conclusion de la phase 1

À l'issue de cette première phase, l'environnement de travail est entièrement opérationnel :

- Système Linux fonctionnel
- Serveur Apache actif
- Site web personnalisé accessible
- Application DVWA correctement déployée
- Base de données configurée

Cette infrastructure constitue une **base réaliste, fonctionnelle et conforme aux exigences académiques**, prête à être utilisée pour la **phase suivante : l'audit de sécurité initial**.

Phase 2 : Audit de sécurité initial (Initial Security Audit)

1. Introduction de la phase

La deuxième phase du projet **Audit et durcissement d'un serveur web** a pour objectif de réaliser un **audit de sécurité initial**, c'est-à-dire une analyse de l'état du serveur **avant toute action de durcissement**.

Cette phase consiste à prendre une **photographie précise et objective** de la situation actuelle :

- Quels services sont actifs ?
- Quels ports sont exposés ?
- Quelles informations sont visibles depuis l'extérieur ?
- Quelles configurations par défaut ou faiblesses potentielles peuvent être identifiées ?

Les résultats obtenus durant cette phase serviront de **référence comparative** lors de l'audit post-durcissement, afin de démontrer l'efficacité des mesures de sécurité appliquées ultérieurement.

2. Organisation du travail et des résultats

Afin de garantir une démarche structurée et exploitable pour le rapport, un répertoire dédié est créé pour stocker l'ensemble des résultats de cette phase.

```shell
mkdir -p ~/web-audit/phase2/{host,network,web,vuln}
cd ~/web-audit/phase2
```

Cette organisation permet de séparer clairement :

- Les informations locales du système
- Les résultats de reconnaissance réseau
- Les données liées au serveur web
- Les scans de vulnérabilités
- **- Terminal montrant la création de l'arborescence `phase2` et de ses sous-dossiers**



3. Audit local du système et des services (Local Audit)

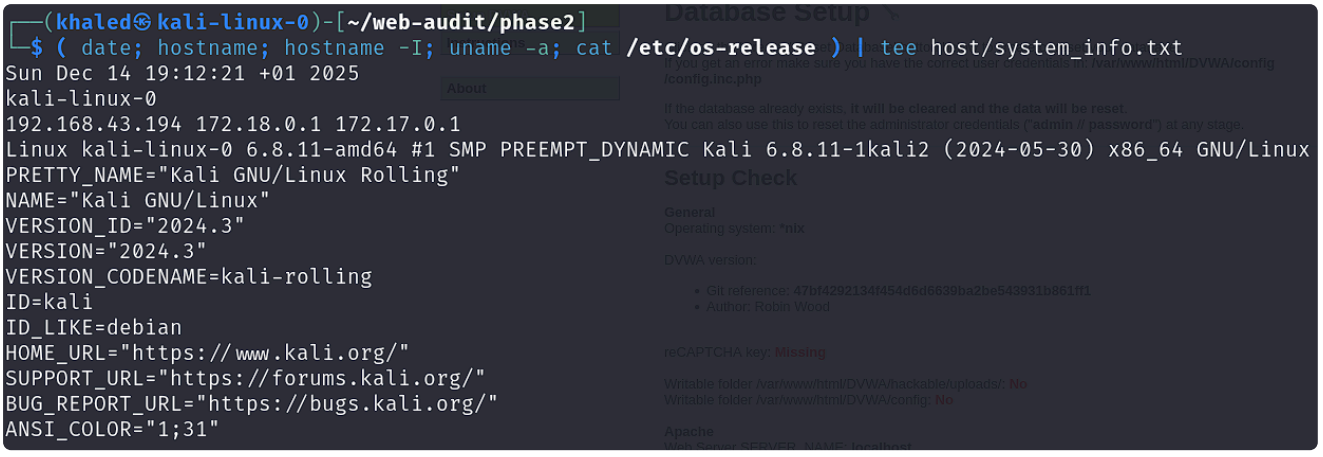
3.1 Identification du système

Les informations générales du système sont collectées afin de documenter précisément l'environnement audité.

```shell
( date; hostname; hostname -I; uname -a; cat /etc/os-release ) | tee host/system_info.txt
```

Ces informations incluent :

- Date de l'audit
- Nom de la machine
- Adresse IP
- Version du noyau
- Distribution Linux utilisée
- **Terminal affichant les informations système collectées**



3.2 Services actifs et ports à l'écoute

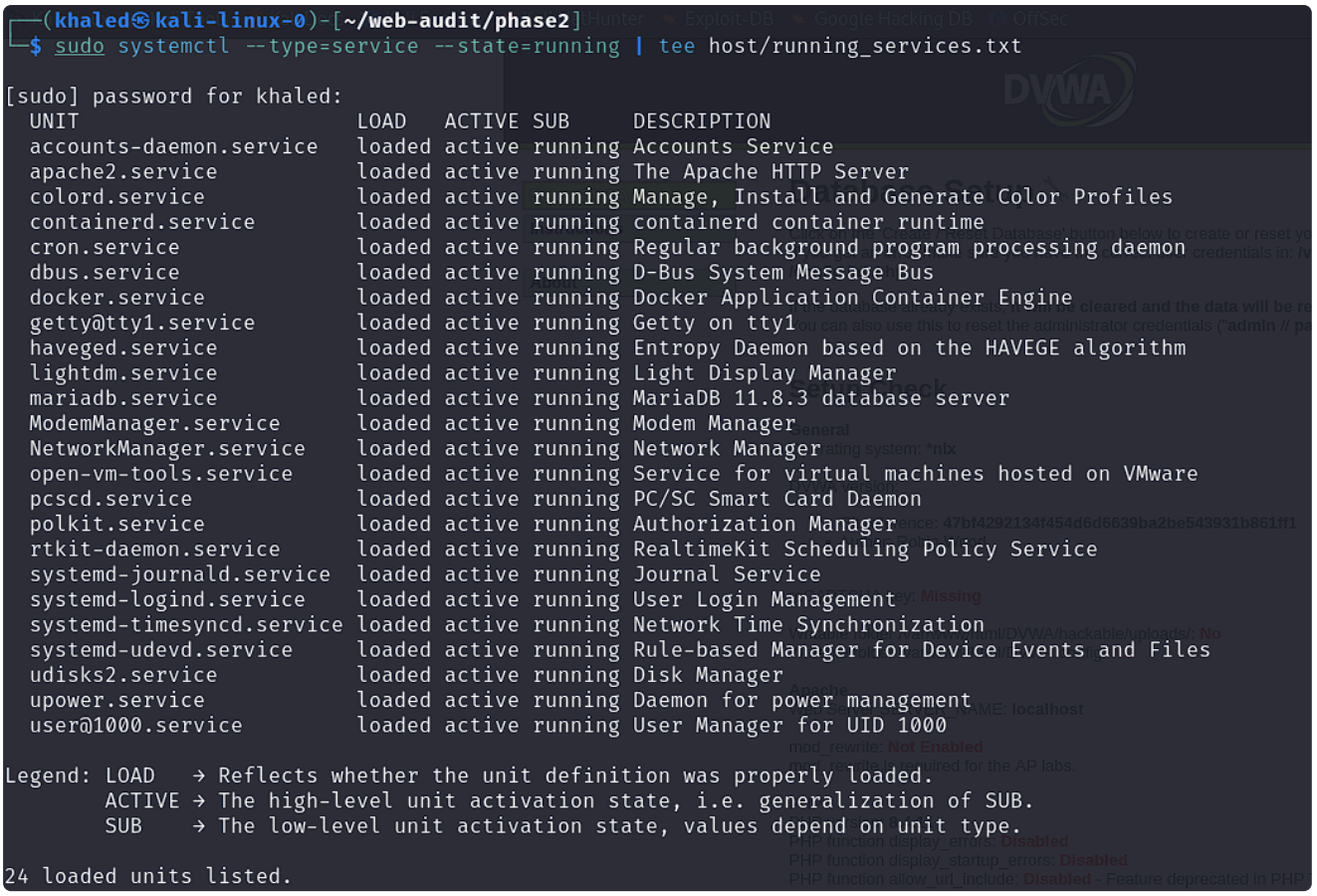
La liste des services en cours d'exécution ainsi que les ports ouverts localement est ensuite identifiée.

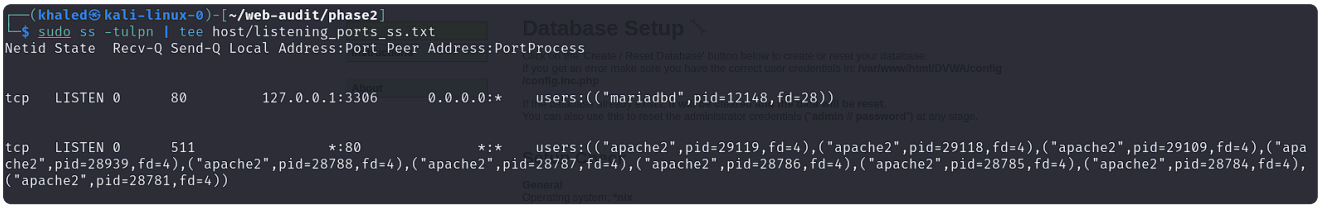
```shell
sudo systemctl --type=service --state=running | tee host/running_services.txt
sudo ss -tulpn | tee host/listening_ports_ss.txt
```

Cette étape permet d'identifier :

- Les services réellement actifs
- Les ports exposés sur le système
- Les processus associés à chaque port

- **Terminal montrant les services actifs && Sortie de la commande `ss -tulpn` indiquant les ports ouverts**





3.3 Versions des services principaux

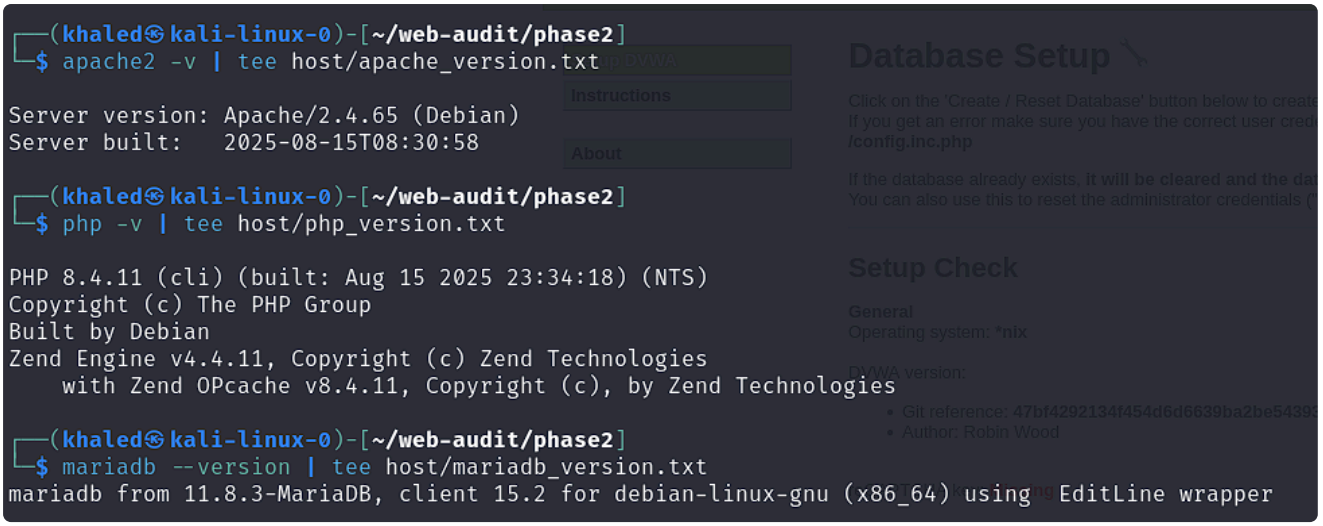
Les versions des services critiques sont collectées, car elles peuvent révéler des informations sensibles à un attaquant.

```shell
apache2 -v | tee host/apache_version.txt
php -v | tee host/php_version.txt
mariadb --version | tee host/mariadb_version.txt
```

Ces informations seront utilisées ultérieurement pour :

- Identifier les versions exposées
- Vérifier si des informations inutiles sont divulguées

- Terminal affichant les versions d'Apache, PHP et MariaDB



3.4 Analyse rapide de la configuration Apache

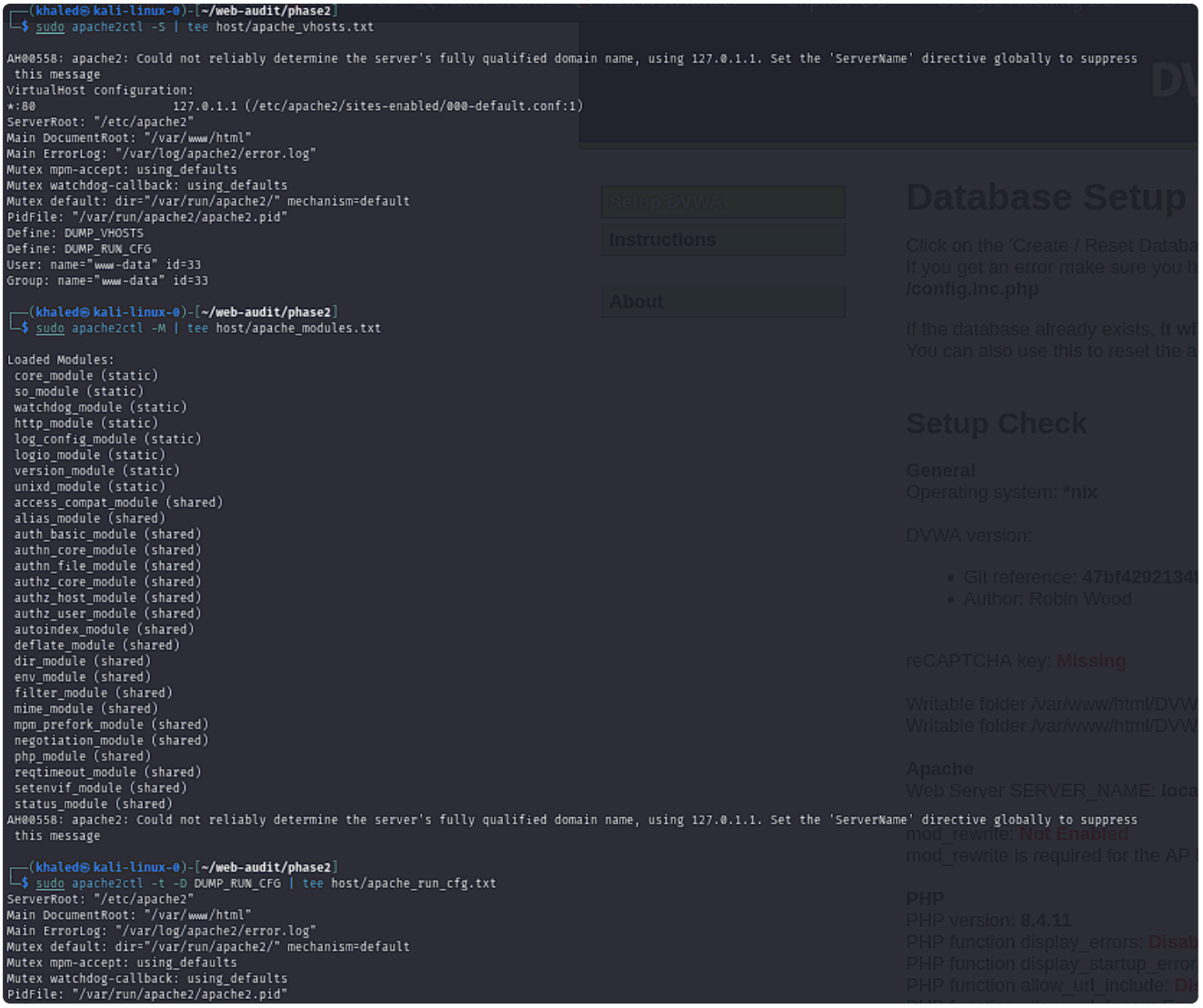
Une lecture synthétique de la configuration Apache est effectuée afin de mieux comprendre sa structure et ses modules actifs.

```shell
sudo apache2ctl -S | tee host/apache_vhosts.txt
sudo apache2ctl -M | tee host/apache_modules.txt
sudo apache2ctl -t -D DUMP_RUN_CFG | tee host/apache_run_cfg.txt
```

Cette étape permet d'identifier :

- Les Virtual Hosts configurés
- Les modules Apache chargés
- Les paramètres globaux de fonctionnement

- Terminal montrant la liste des modules Apache chargés



---

4. Reconnaissance réseau externe (Network Reconnaissance)

Dans cette partie, le serveur est analysé **du point de vue d'un attaquant externe**.

L'adresse IP cible est stockée dans une variable pour faciliter les scans.

```shell
TARGET="PUT_TARGET_IP_HERE"
echo "$TARGET" | tee network/target_ip.txt
```

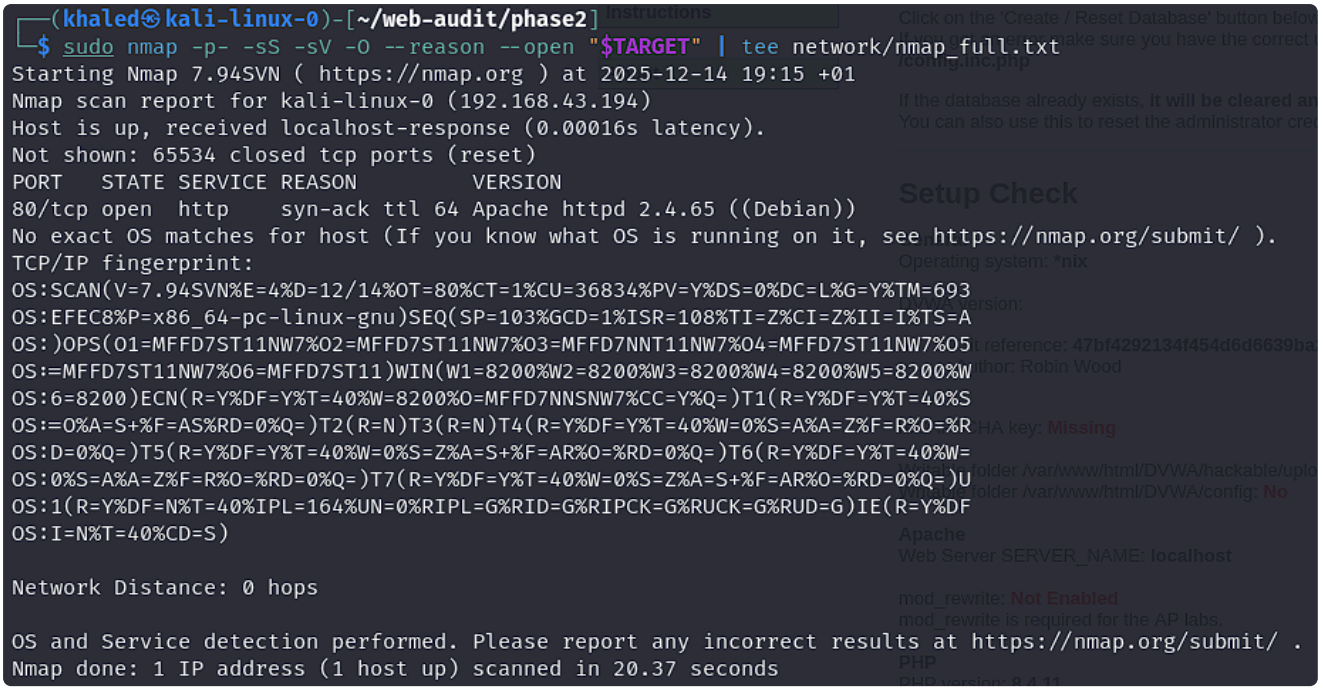
4.1 Scan complet des ports avec Nmap

Un scan complet des ports TCP est réalisé afin d'identifier tous les services exposés.

```shell
sudo nmap -p- -sS -sV -O --reason --open "$TARGET" | tee network/nmap_full.txt
```

Ce scan permet d'identifier :

- Les ports ouverts
- Les services associés
- Les versions détectées
- Le système d'exploitation estimé
- **Extrait du scan Nmap montrant les ports ouverts et le service Apache**



4.2 Scan Nmap ciblé sur les services web

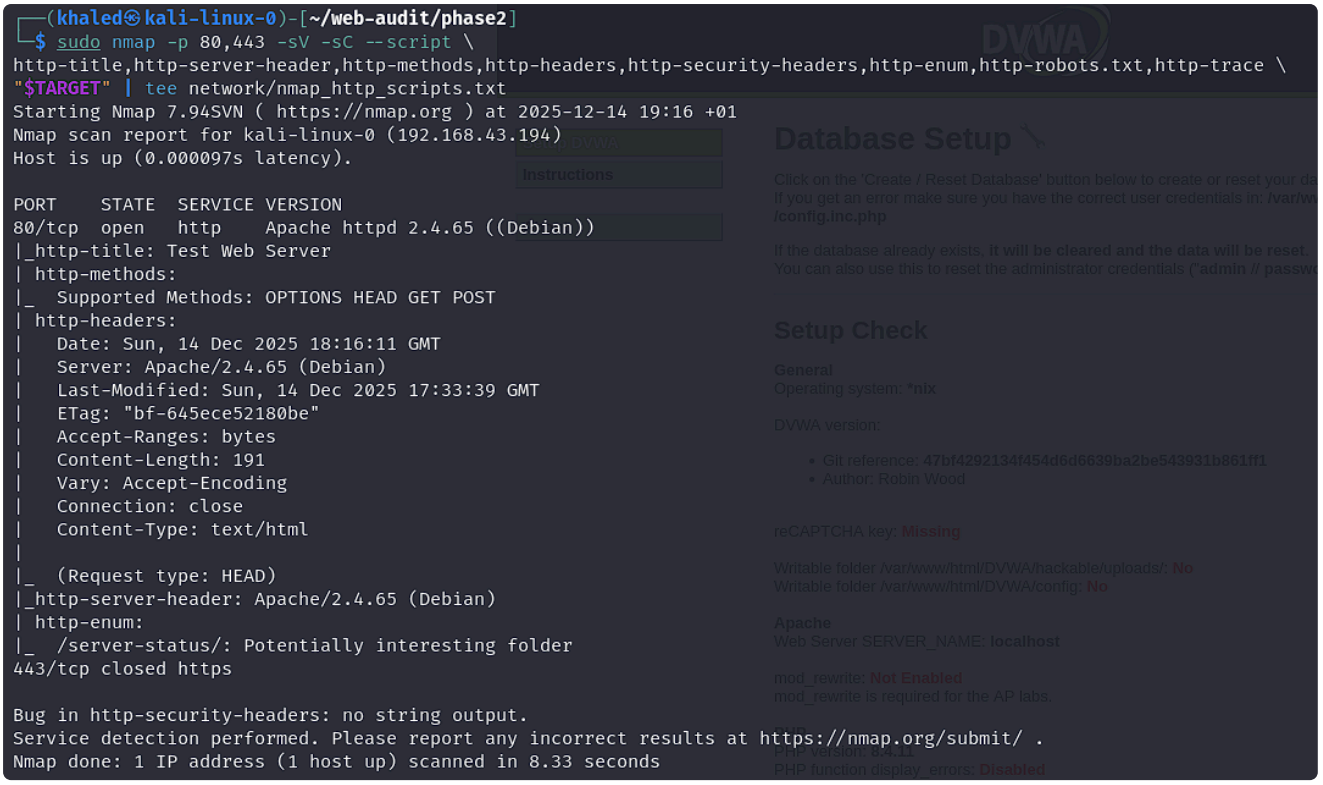
Un scan plus spécifique est ensuite effectué sur les ports web avec des scripts HTTP.

```shell
sudo nmap -p 80,443 -sV -sC --script \
http-title,http-server-header,http-methods,http-headers,http-security-headers,http-enum,http-robots.txt,http-trace \
"$TARGET" | tee network/nmap_http_scripts.txt
```

Ce scan permet de détecter :

- Les méthodes HTTP autorisées
- Les en-têtes de sécurité manquants
- Les informations révélées par le serveur web

- Résultat Nmap indiquant les méthodes HTTP et les headers HTTP détectés



---

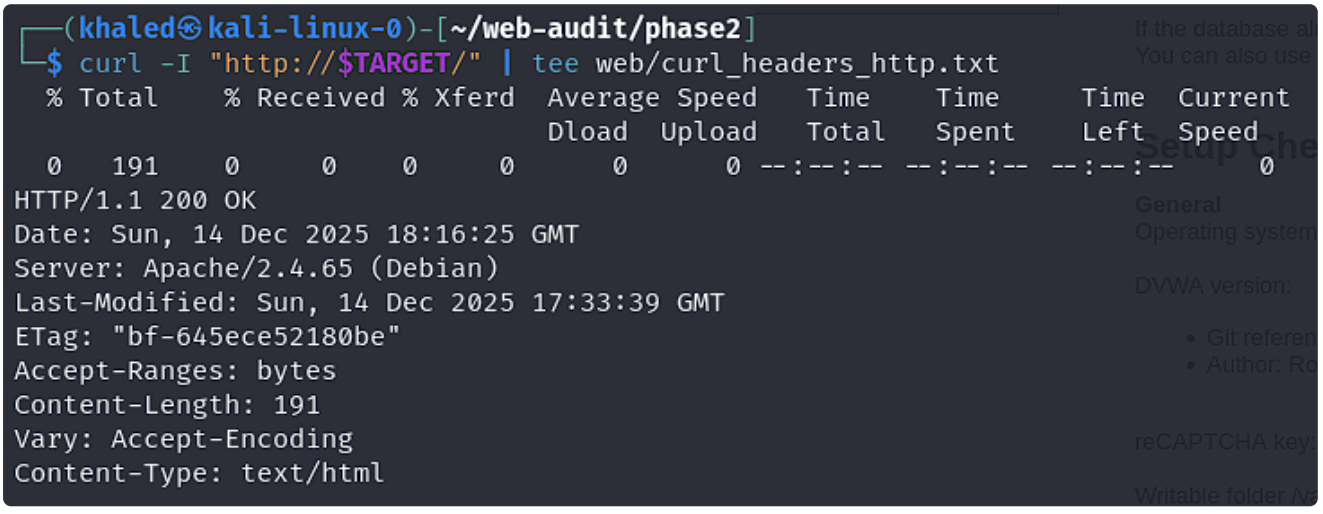
5. Empreinte du serveur web (Web Fingerprinting)

5.1 Analyse des en-têtes HTTP

Les en-têtes HTTP retournés par le serveur sont analysés afin d'identifier les informations exposées.

```shell
curl -I "http://$TARGET/" | tee web/curl_headers_http.txt
```

- Terminal affichant les en-têtes HTTP retournés par le serveur

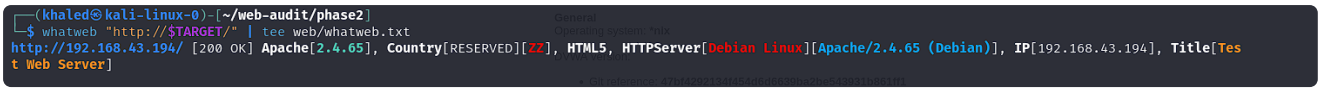


5.2 Identification des technologies utilisées

L'outil WhatWeb est utilisé pour identifier rapidement les technologies visibles.

```shell
whatweb "http://$TARGET/" | tee web/whatweb.txt
```

- Résultat WhatWeb indiquant Apache et les technologies détectées

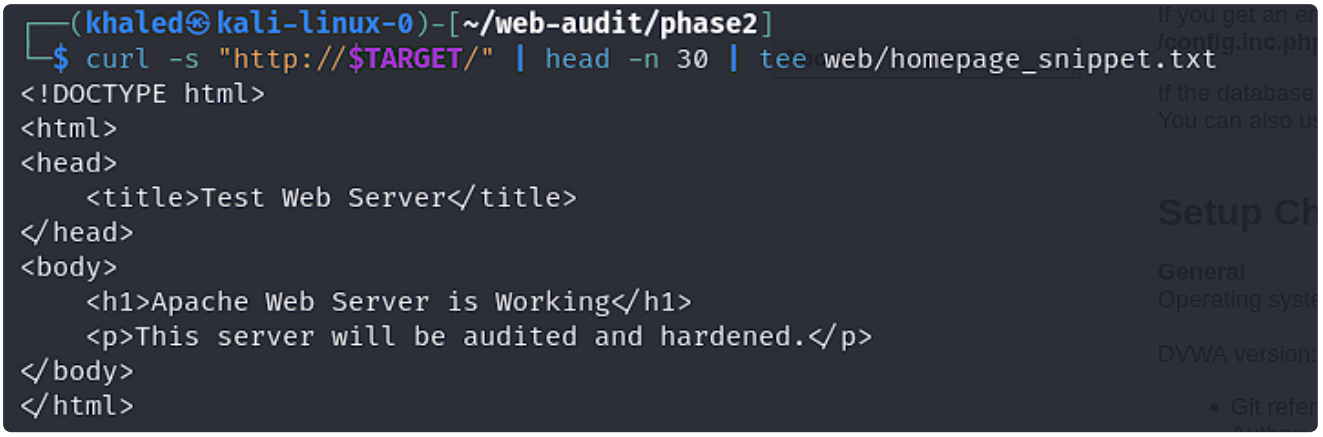


5.3 Vérification du contenu exposé

Un extrait du contenu HTML de la page d'accueil est récupéré.

```shell
curl -s "http://$TARGET/" | head -n 30 | tee web/homepage_snippet.txt
```

- Terminal montrant un extrait du code HTML retourné



---

6. Scan de vulnérabilités web automatisé

6.1 Scan Nikto du serveur web

Nikto est utilisé pour détecter rapidement des problèmes de configuration web.

```shell
nikto -h "http://$TARGET" -output vuln/nikto.txt
```

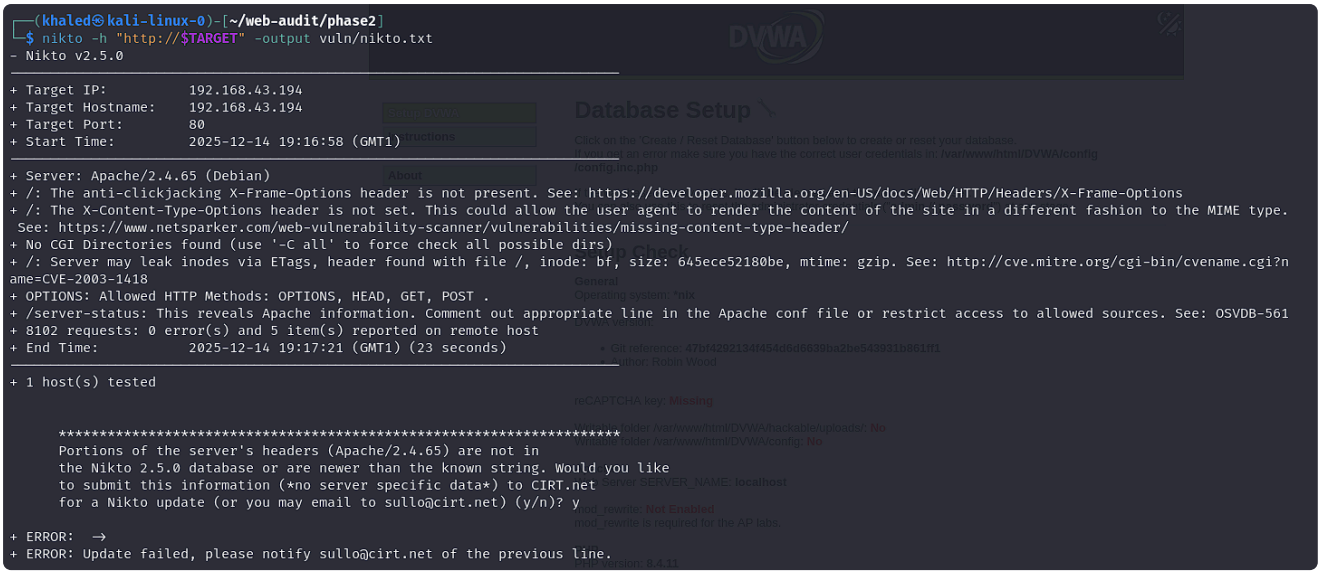
Si l'application DVWA est accessible :

```shell
nikto -h "http://$TARGET/dvwa/" -output vuln/nikto_dvwa.txt
```

Nikto permet de détecter :

- En-têtes de sécurité manquants

- Fichiers ou chemins par défaut
- Configurations web non sécurisées
- Extrait du rapport Nikto montrant des avertissements ou alertes



---

7. Identification des configurations par défaut non sécurisées

À partir des résultats collectés, plusieurs points d'attention sont identifiés :

- Divulgation de la version Apache
- Absence de certains en-têtes de sécurité HTTP
- Méthodes HTTP non nécessaires
- Présence de chemins ou pages par défaut
- Configuration web par défaut sans durcissement

Pour faciliter l'analyse, une extraction rapide des éléments clés est réalisée :

```shell
grep -Ei "Server:|X-|Content-Security-Policy|Strict-Transport-Security|Trace|Options|Methods" -n \
web/curl_headers_http.txt network/nmap_http_scripts.txt 2>/dev/null | tee web/quick_findings.txt
```

- Terminal affichant les résultats extraits dans `quick_findings.txt`



---

8. Critères de réussite de la phase 2

La phase d'audit initial est considérée comme réussie lorsque :

- Les ports et services exposés sont clairement identifiés
- Les versions des services sont documentées
- Les informations visibles depuis l'extérieur sont connues
- Des fichiers de résultats exploitables sont disponibles
- Une base solide de comparaison est établie pour l'audit post-durcissement

---

9. Conclusion de la phase 2

Cette phase a permis d'obtenir une **vision claire et factuelle de l'état de sécurité initial du serveur web**.
Les informations collectées constituent une base essentielle pour :

- Identifier les faiblesses réelles
- Prioriser les actions de durcissement
- Mesurer objectivement les améliorations après sécurisation

La phase suivante consistera à **analyser ces résultats et à évaluer les risques associés**, avant la mise en œuvre des mesures de durcissement.

Phase 3 : Analyse des résultats et évaluation des risques

(Risk Analysis)

---

1. Introduction

Cette phase vise à transformer les résultats techniques bruts obtenus lors de l'audit initial (Phase 2) en une **analyse de sécurité exploitable et compréhensible**.
L'objectif n'est pas d'identifier un maximum de failles, mais de **qualifier les risques réels**, d'en estimer la gravité et de **définir des priorités de durcissement** adaptées à un projet académique de niveau intermédiaire.

Les sources analysées incluent notamment les résultats de Nmap, Nikto, curl, WhatWeb ainsi que l'examen des configurations d'Apache, PHP et MariaDB.

---

2. Vue d'ensemble du système audité

Le serveur audité présente les caractéristiques suivantes :

- Système d'exploitation : Kali GNU/Linux Rolling 2024.3
- Environnement : Machine virtuelle (VMware)
- Serveur web : Apache HTTP Server 2.4.65

- Langage côté serveur : PHP 8.4.11
- Base de données : MariaDB 11.8.3
- Adresse IP : 192.168.43.194

Le serveur est utilisé à des fins **éducatives et expérimentales**, ce qui oriente les choix de priorisation des risques.

---

3. Analyse des services et des ports exposés

Résultats principaux

- Port **80/tcp (HTTP)** ouvert et accessible
- Port **443/tcp (HTTPS)** non actif
- MariaDB accessible uniquement en local (127.0.0.1)
- Absence d'autres services exposés sur le réseau

Évaluation du risque

La surface d'attaque réseau est **limitée**, ce qui constitue un point positif.
Cependant, l'absence de HTTPS implique que les communications HTTP ne sont pas protégées.

Niveau de risque global : Moyen

---

4. Analyse de la configuration Apache

Constats

- La version exacte d'Apache est divulguée via les en-têtes HTTP
- Le paramètre `ServerName` n'est pas explicitement défini
- Le chemin `/server-status` est accessible
- Plusieurs modules Apache sont activés par défaut, sans nécessité fonctionnelle immédiate

Risques associés

- La divulgation de version facilite l'exploitation de vulnérabilités connues
- L'accès à `server-status` expose des informations internes sur l'état du serveur
- Les modules inutiles augmentent inutilement la surface d'attaque

Niveau de risque : Moyen à Élevé (selon l'élément)

---

5. Analyse des en-têtes HTTP

En-têtes de sécurité manquants

Les analyses ont révélé l'absence de plusieurs en-têtes de sécurité importants :

- X-Frame-Options
- X-Content-Type-Options
- Content-Security-Policy
- Strict-Transport-Security (HSTS)

Impact sécuritaire

- Risque de Clickjacking
- Vulnérabilité au MIME sniffing
- Absence de protection contre certaines attaques XSS
- Protection insuffisante du canal de communication

Niveau de risque : Moyen

---

6. Analyse de l'application web

État actuel

- Page HTML statique simple
- Absence de formulaires ou d'entrées utilisateur
- Aucun traitement dynamique côté serveur

Évaluation

Dans son état actuel, l'application web ne présente pas de vulnérabilités applicatives directes telles que SQL Injection ou Cross-Site Scripting.
Les risques observés sont liés principalement à la **configuration du serveur**, et non au contenu applicatif.

Niveau de risque : Faible

---

7. Synthèse des résultats Nikto

Les principaux constats issus du scan Nikto confirment :

- L'absence d'en-têtes de sécurité HTTP
- La présence de configurations par défaut
- La divulgation d'informations via ETag
- L'activation de méthodes HTTP non essentielles
- L'exposition du chemin `/server-status`

Ces résultats corroborent les observations faites via Nmap et curl.

---

8. Classification des vulnérabilités

A. Vulnérabilités de configuration

- Divulgation de la version Apache
- En-têtes HTTP de sécurité manquants
- Configuration par défaut non durcie

B. Exposition de services

- Utilisation de HTTP sans HTTPS
- Méthodes HTTP inutiles activées

C. Vulnérabilités applicatives

- Aucune vulnérabilité applicative critique identifiée à ce stade

---

9. Définition des priorités

Compte tenu du cadre académique et du niveau du projet, les actions à prioriser sont :

- Durcissement de la configuration Apache
- Réduction des informations divulguées
- Ajout des en-têtes de sécurité HTTP
- Désactivation des fonctionnalités inutiles
- Mise en place de HTTPS

Les tests d'intrusion applicatifs avancés et les configurations de niveau production ne sont pas inclus dans le périmètre.

---

10. Conclusion de la phase 3

L'analyse des résultats montre que le serveur web est **fonctionnel mais insuffisamment durci**. Les risques identifiés sont majoritairement liés à des **configurations par défaut** et à des pratiques courantes mais non sécurisées.

Cette situation constitue une base idéale pour la phase suivante, qui consistera à **appliquer des mesures de durcissement ciblées** et à démontrer concrètement l'amélioration du niveau de sécurité du serveur.

Phase 4 : Durcissement du système d'exploitation (System Hardening)

---

1. Introduction de la phase

Le durcissement du système d'exploitation constitue une étape fondamentale avant toute sécurisation du serveur web ou de l'application.
L'objectif principal de cette phase est de **réduire la surface d'attaque (attack surface)** du serveur en conservant uniquement les composants strictement nécessaires au fonctionnement du service web.

Le principe appliqué est le suivant :

> Un serveur web n'a besoin que d'un nombre limité de services ; toute fonctionnalité supplémentaire représente un risque potentiel.

Cette phase adopte une approche **claire, pragmatique et adaptée à un contexte académique de niveau intermédiaire**, en mettant en œuvre des mesures simples mais efficaces.

---

2. Mise à jour du système et des paquets

Objectif

Corriger les vulnérabilités connues du système d'exploitation et des logiciels installés.

Commandes exécutées

```shell
sudo apt update
sudo apt full-upgrade -y
sudo apt autoremove -y
sudo apt autoclean
```

Résultats

- Mise à jour du noyau et des paquets système
- Suppression des dépendances obsolètes ou inutilisées
- Amélioration globale de la stabilité et de la sécurité du système

Cette étape est essentielle et doit être explicitement mentionnée dans tout audit ou processus de durcissement.

---

3. Désactivation des services non nécessaires

Objectif

Limiter le nombre de services actifs afin de réduire les points d'entrée potentiels.

3.1 Identification des services actifs

```shell
sudo systemctl --type=service --state=running
```

Cette commande permet d'identifier les services réellement en fonctionnement sur le système.

3.2 Services nécessaires au projet

Dans le cadre de ce projet, les services considérés comme nécessaires sont :

- apache2
- mariadb (si conservé pour l'application DVWA)
- NetworkManager
- services systemd essentiels
- open-vm-tools (environnement VMware)

Tous les autres services sont évalués en fonction de leur utilité réelle.

3.3 Désactivation de services non essentiels

Certains services, non requis dans un contexte académique de serveur web, peuvent être désactivés sans impact fonctionnel.

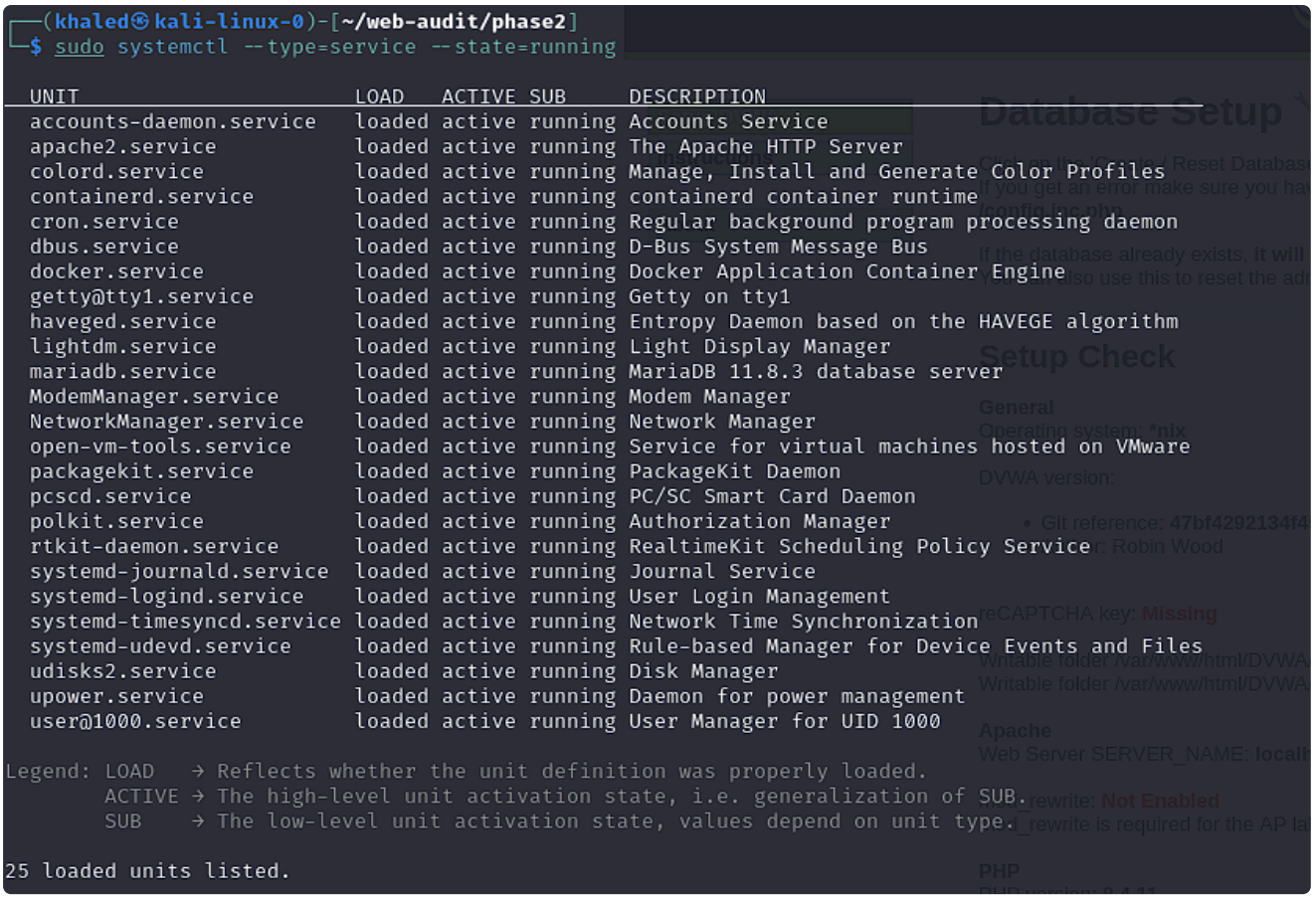
```shell
sudo systemctl disable bluetooth --now 2>/dev/null
sudo systemctl disable cups --now 2>/dev/null
sudo systemctl disable avahi-daemon --now 2>/dev/null
```

Ces actions permettent de limiter l'exposition inutile du système.

Vérification

```shell
sudo systemctl --type=service --state=running
```

- **Liste des services actifs après désactivation && Comparaison visuelle avec l'état initial :**



```
┌──(khaled㉿kali-linux-0)-[~/web-audit/phase2]
└─$ sudo systemctl --type=service --state=running

  UNIT                        LOAD   ACTIVE SUB     DESCRIPTION
  accounts-daemon.service     loaded active running Accounts Service
  apache2.service             loaded active running The Apache HTTP Server
  colord.service              loaded active running Manage, Install and Generate Color Profiles
  containerd.service          loaded active running containerd container runtime
  cron.service                loaded active running Regular background program processing daemon
  dbus.service                loaded active running D-Bus System Message Bus
  docker.service              loaded active running Docker Application Container Engine
  getty@tty1.service          loaded active running Getty on tty1
  haveged.service             loaded active running Entropy Daemon based on the HAVEGE algorithm
  lightdm.service             loaded active running Light Display Manager
  mariadb.service             loaded active running MariaDB 11.8.3 database server
  ModemManager.service        loaded active running Modem Manager
  NetworkManager.service      loaded active running Network Manager
  open-vm-tools.service       loaded active running Service for virtual machines hosted on VMware
  packagekit.service          loaded active running PackageKit Daemon
  pcscd.service               loaded active running PC/SC Smart Card Daemon
  polkit.service              loaded active running Authorization Manager
  rtkit-daemon.service        loaded active running RealtimeKit Scheduling Policy Service
  systemd-journald.service    loaded active running Journal Service
  systemd-logind.service      loaded active running User Login Management
  systemd-timesyncd.service loaded active running Network Time Synchronization
  systemd-udevd.service       loaded active running Rule-based Manager for Device Events and Files
  udisks2.service             loaded active running Disk Manager
  upower.service              loaded active running Daemon for power management
  user@1000.service           loaded active running User Manager for UID 1000

Legend: LOAD   → Reflects whether the unit definition was properly loaded.
        ACTIVE → The high-level unit activation state, i.e. generalization of SUB.
        SUB    → The low-level unit activation state, values depend on unit type.

25 loaded units listed.
```

---

4. Configuration du pare-feu (Firewall – UFW)

Objectif

Contrôler précisément les flux réseau entrants et sortants.

4.1 Installation de UFW

```shell
sudo apt install ufw -y
```

4.2 Définition des politiques par défaut

```shell
sudo ufw default deny incoming
sudo ufw default allow outgoing
```

Ces règles garantissent que **tout trafic entrant non autorisé est bloqué par défaut**.

4.3 Autorisation des ports nécessaires

```shell
sudo ufw allow 80/tcp
sudo ufw allow 22/tcp
```

Les ports ouverts correspondent uniquement :

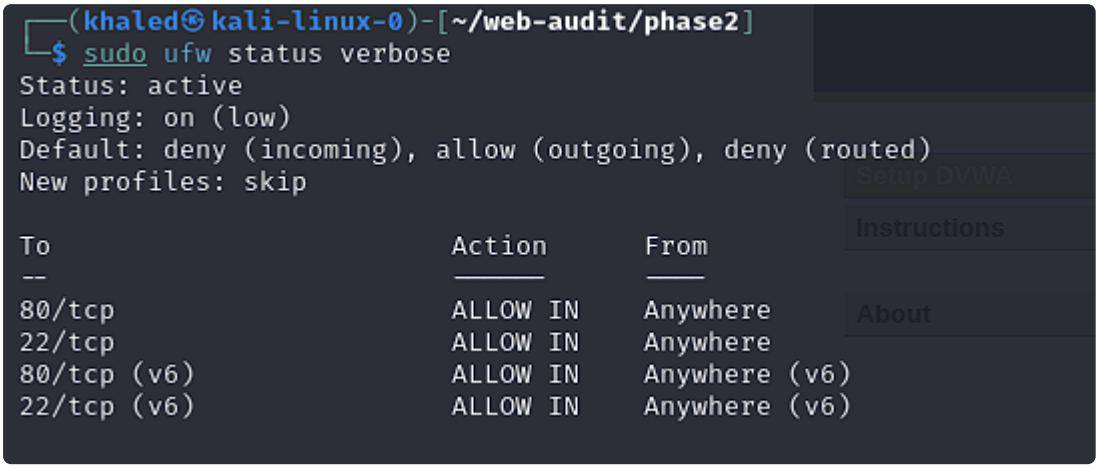
- au service web (HTTP)
- à l'administration via SSH (à des fins académiques)

4.4 Activation et vérification

```shell
sudo ufw enable
sudo ufw status verbose
```

Résultat attendu :

- Ports autorisés : 80/tcp, 22/tcp
- Tous les autres ports bloqués



---

5. Gestion des utilisateurs et des permissions

Objectif

Limiter l'impact d'une compromission de compte utilisateur.

5.1 Désactivation du compte root pour la connexion directe

```shell
sudo passwd -l root
```

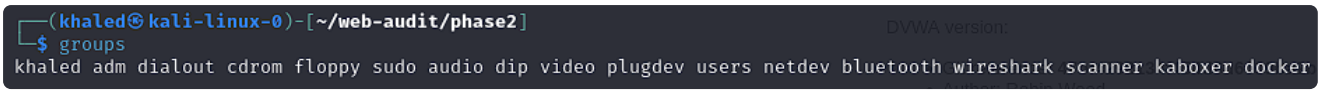
Cette mesure empêche l'authentification directe en tant que superutilisateur.

5.2 Vérification de l'appartenance au groupe sudo

```shell
groups
```

Le compte utilisateur doit appartenir au groupe `sudo`.

- Terminal montrant l'appartenance au groupe sudo



5.3 Vérification des permissions du répertoire web

```shell
ls -ld /var/www/html
```

Permissions attendues :

```
drwxr-xr-x root root
```

Si nécessaire :

```shell
sudo chown -R root:root /var/www/html
sudo chmod -R 755 /var/www/html
```

---

6. Sécurisation de l'accès SSH

Objectif

Limiter les attaques par force brute et les accès non autorisés.

6.1 Modification de la configuration SSH

```shell
sudo nano /etc/ssh/sshd_config
```

Paramètres appliqués :

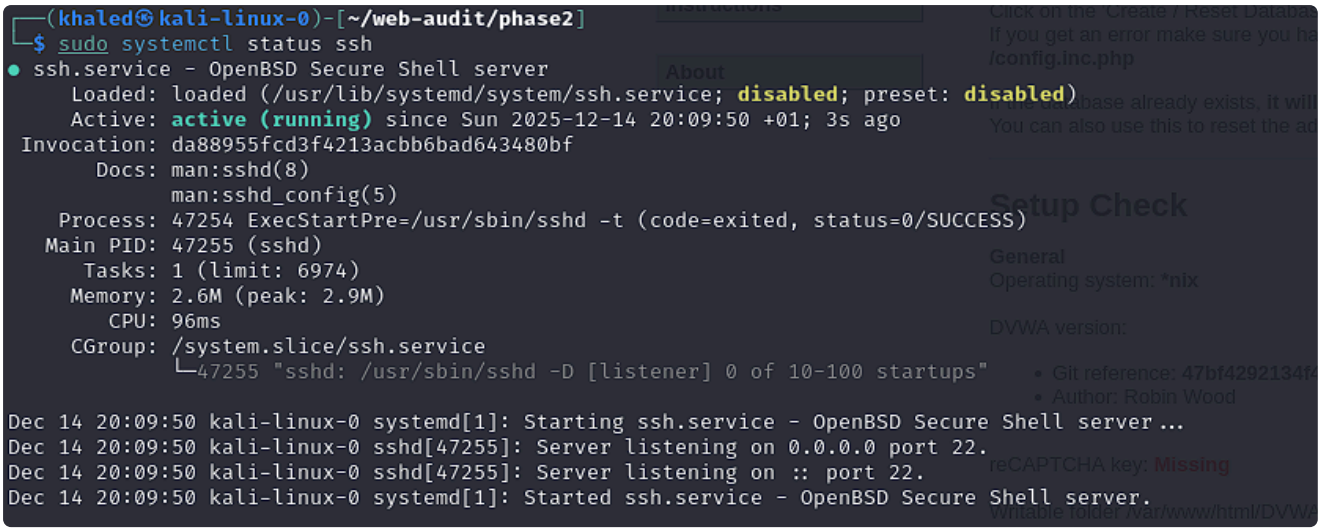
```
PermitRootLogin no
PasswordAuthentication yes
MaxAuthTries 3
```

L'authentification par mot de passe est conservée pour des raisons pédagogiques.

6.2 Redémarrage et vérification

```shell
sudo systemctl restart ssh
sudo systemctl status ssh
```

- État du service SSH après redémarrage



---

7. Vérification finale du durcissement système

Commandes de contrôle

```shell
sudo ufw status
sudo systemctl --type=service --state=running
ss -tulpn
```

Résultat attendu

- Nombre réduit de services actifs
- Ports réseau limités aux services essentiels
- Apache fonctionnel sans exposition inutile

```
To                         Action      From
--                         ------      ----
80/tcp                     ALLOW       Anywhere
22/tcp                     ALLOW       Anywhere
80/tcp (v6)                ALLOW       Anywhere (v6)
22/tcp (v6)                ALLOW       Anywhere (v6)
```

---

8. Conclusion de la phase 4

À l'issue de cette phase, le système d'exploitation a été significativement durci :

- Système à jour

- Services non essentiels désactivés
- Pare-feu configuré selon le principe du moindre privilège
- Gestion des utilisateurs et des permissions renforcée
- Accès SSH sécurisé de manière basique

Le serveur dispose désormais d'une **base système robuste**, prête pour la phase suivante dédiée au **durcissement spécifique du serveur web Apache**.

Phase 5 : Durcissement du serveur web Apache

(Web Server Hardening)

---

1. Introduction de la phase

Après le durcissement du système d'exploitation, la sécurisation du serveur web constitue une étape essentielle du projet **Audit et durcissement d'un serveur web**.
Dans la majorité des cas, les attaques contre les serveurs web exploitent **des configurations par défaut faibles** ou **des informations inutilement exposées**, plutôt que des vulnérabilités complexes.

L'objectif de cette phase est donc de :

- Réduire les informations divulguées par Apache
- Diminuer la surface d'attaque du serveur web
- Corriger les mauvaises pratiques courantes (Directory Listing, modules inutiles, méthodes HTTP dangereuses)

Les mesures appliquées restent **simples, compréhensibles et adaptées à un contexte académique de niveau intermédiaire**.

---

2. Masquage des informations du serveur (Server Information Disclosure)

Problème identifié

Par défaut, Apache divulgue :

- Le type de serveur
- Le numéro de version
- Parfois des informations liées au système

Ces éléments facilitent le **fingerprinting** et aident un attaquant à cibler des vulnérabilités connues.

Mesure appliquée

Modification du fichier de configuration de sécurité :

```shell
sudo nano /etc/apache2/conf-enabled/security.conf
```

Paramètres définis :

```
ServerTokens Prod
ServerSignature Off
```

Interprétation

- `ServerTokens Prod` limite l'information envoyée au strict minimum
- `ServerSignature Off` supprime les informations du serveur dans les pages d'erreur

---

Application et vérification

```shell
sudo systemctl reload apache2
```

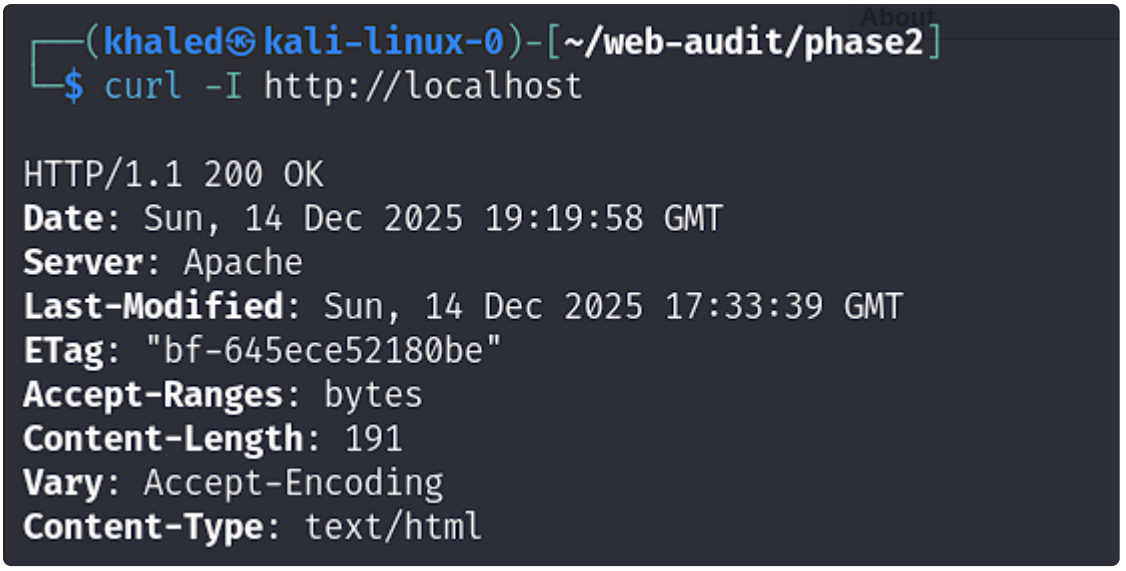
Vérification :

```shell
curl -I http://localhost
```

Le header doit contenir uniquement :

```
Server: Apache
```

- Terminal affichant le résultat de `curl -I` && Absence du numéro de version Apache :



3. Désactivation du Directory Listing

Problème identifié

En l'absence de fichier `index`, Apache peut afficher la **liste des fichiers d'un répertoire**, exposant potentiellement des données sensibles.

Mesure appliquée

Édition du Virtual Host par défaut :

```shell
sudo nano /etc/apache2/sites-available/000-default.conf
```

Ajout ou vérification de la directive suivante :

```
<Directory /var/www/html>
    Options -Indexes
</Directory>
```

Application et vérification

```shell
sudo systemctl reload apache2
```

Tentative d'accès à un répertoire sans fichier index :
→ La liste des fichiers ne doit plus apparaître.

---

4. Désactivation des modules Apache non nécessaires

Principe

Chaque module Apache actif représente une **surface d'attaque supplémentaire**.
Dans un contexte académique, seuls les modules strictement nécessaires doivent être conservés.

Identification des modules chargés

```shell
apache2ctl -M
```

Modules désactivés

```shell
sudo a2dismod status
sudo a2dismod autoindex
```

Justification

- `status` : expose des informations internes sur le serveur
- `autoindex` : responsable de l'affichage des listes de fichiers

Application

```shell
sudo systemctl reload apache2
```

---

5. Renforcement des permissions des fichiers web

Objectif

Limiter les droits d'accès afin de :

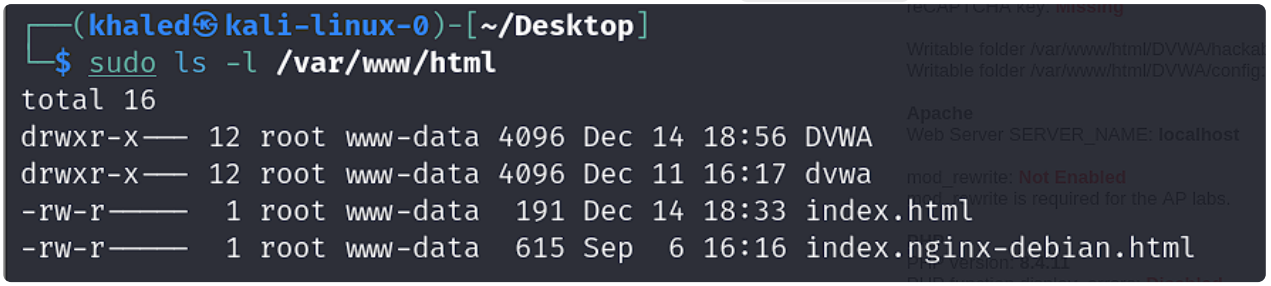
- Empêcher les écritures non nécessaires
- Réduire l'impact d'une compromission du serveur web

Permissions appliquées

```shell
sudo chown -R root:www-data /var/www/html
sudo find /var/www/html -type d -exec chmod 750 {} \;
sudo find /var/www/html -type f -exec chmod 640 {} \;
```

Interprétation

- Répertoires : accès lecture/exécution contrôlé
- Fichiers : lecture uniquement pour Apache
- Apache fonctionne avec l'utilisateur `www-data`
- Terminal affichant les permissions finales du répertoire `/var/www/html`



---

6. Désactivation de la méthode HTTP TRACE

Problème identifié

La méthode HTTP TRACE peut être exploitée dans des attaques de type **Cross-Site Tracing (XST)**.

Mesure appliquée

Édition de la configuration globale Apache :

```shell
sudo nano /etc/apache2/apache2.conf
```

Ajout de la directive :

```
TraceEnable Off
```

Application et vérification

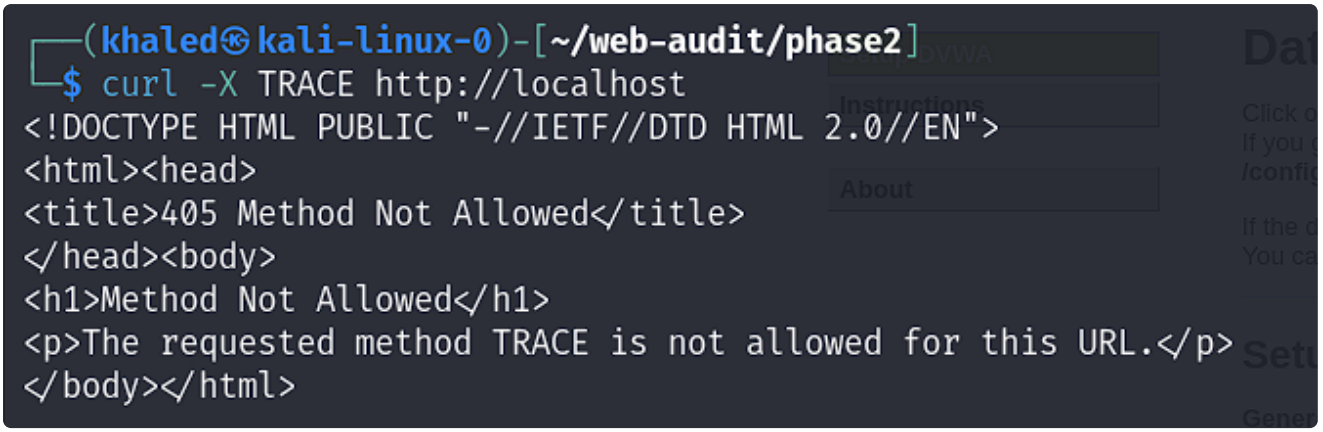
```shell
sudo systemctl reload apache2
```

Test :

```shell
curl -X TRACE http://localhost
```

La requête ne doit pas être acceptée.

- Terminal montrant l'échec de la requête TRACE



---

7. Protection basique contre les attaques de type DoS

Objectif

Limiter les requêtes lentes ou abusives de manière simple et académique.

Mesure appliquée

Modification du fichier de sécurité Apache :

```shell
sudo nano /etc/apache2/conf-enabled/security.conf
```

Ajout de la directive suivante :

```
RequestReadTimeout header=20-40,MinRate=500 body=20,MinRate=500
```

Cette règle empêche les connexions excessivement lentes.

Application

```shell
sudo systemctl reload apache2
```

---

8. Bilan de la phase 5

À l'issue de cette phase, le serveur Apache a été significativement durci :

- Informations du serveur masquées
- Directory Listing désactivé
- Modules inutiles supprimés
- Permissions des fichiers renforcées
- Méthode HTTP TRACE désactivée
- Protection basique contre les attaques DoS

Ces actions répondent directement aux **faiblesses identifiées lors de la phase 3**, et constituent un **durcissement web clair, efficace et académiquement pertinent**.

---

9. Conclusion

Le serveur web Apache est désormais configuré selon le principe du **moindre privilège**, avec une exposition minimale des informations et des fonctionnalités.
Cette étape prépare le terrain pour la phase suivante, dédiée à la **sécurisation des communications (HTTPS / TLS et en-têtes de sécurité)**.

Phase 6 : Sécurisation des communications (HTTPS / TLS)

---

1. Introduction de la phase

La sécurisation des communications est une composante essentielle de tout serveur web moderne.
L'utilisation du protocole **HTTPS** permet de garantir la **confidentialité**, l'**intégrité** et, dans une certaine mesure, l'**authenticité** des données échangées entre le client et le serveur.

Dans un cadre académique, l'utilisation d'un **certificat TLS auto-signé (Self-Signed)** est suffisante pour démontrer la compréhension du mécanisme HTTPS et sa mise en œuvre correcte.
L'objectif de cette phase n'est pas d'atteindre un niveau de production avancé, mais de configurer Apache avec :

- Un support HTTPS fonctionnel
- Des protocoles de chiffrement sûrs
- La désactivation des versions obsolètes et vulnérables

---

2. Activation du module SSL dans Apache

Objectif

Permettre à Apache de gérer les connexions HTTPS.

Actions réalisées

Activation du module SSL :

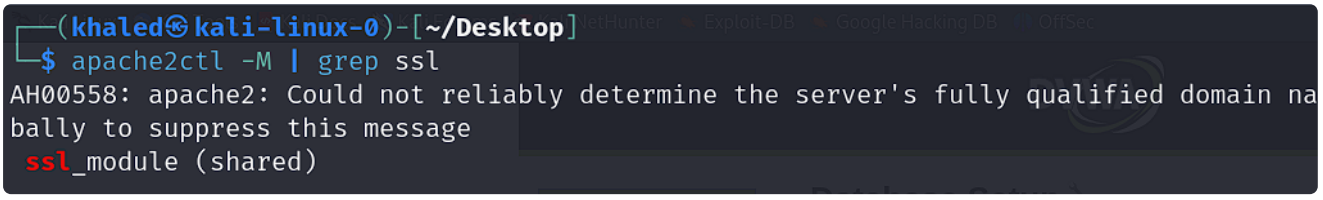
```shell
sudo a2enmod ssl
sudo systemctl restart apache2
```

Vérification du chargement du module :

```shell
apache2ctl -M | grep ssl
```

La présence de `ssl_module` confirme l'activation correcte.

- Terminal montrant le résultat de `apache2ctl -M | grep ssl` && La ligne `ssl_module` doit être visible



---

3. Création d'un certificat TLS auto-signé

Objectif

Générer un certificat SSL valide pour les besoins du projet.

Création du répertoire de stockage

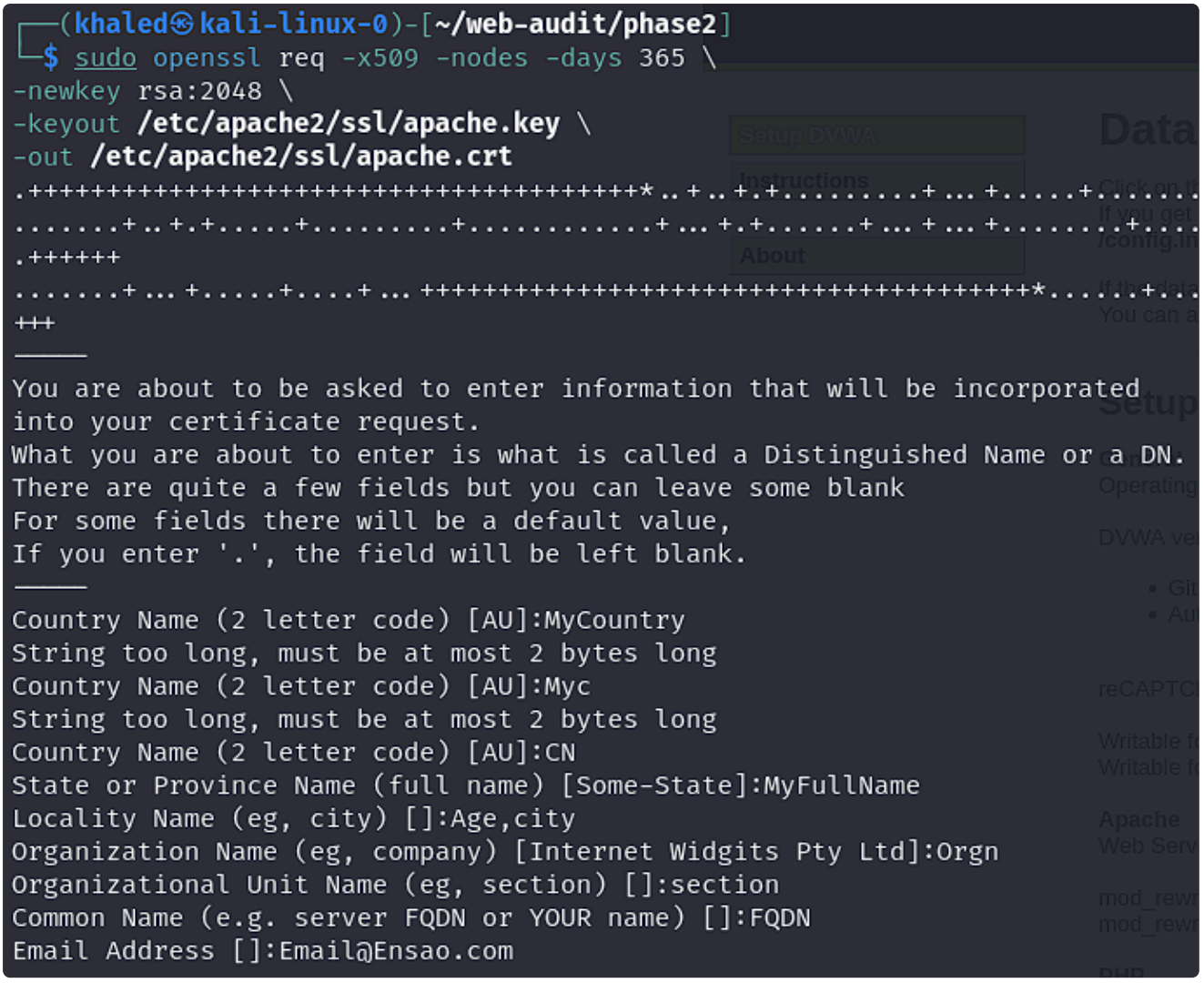
```shell
sudo mkdir -p /etc/apache2/ssl
```

Génération du certificat et de la clé privée

```shell
sudo openssl req -x509 -nodes -days 365 \
-newkey rsa:2048 \
-keyout /etc/apache2/ssl/apache.key \
-out /etc/apache2/ssl/apache.crt
```

Lors de la génération :

- Les champs peuvent contenir des valeurs arbitraires
- Le **Common Name** est défini comme `localhost` ou l'adresse IP du serveur
- **Terminal montrant la génération du certificat et de la clé privée**



---

4. Activation du VirtualHost HTTPS

Objectif

Configurer Apache pour accepter les connexions HTTPS sur le port 443.

Activation du site SSL par défaut

```shell
sudo a2ensite default-ssl
```

Configuration du VirtualHost HTTPS

Édition du fichier :

```shell
sudo nano /etc/apache2/sites-available/default-ssl.conf
```

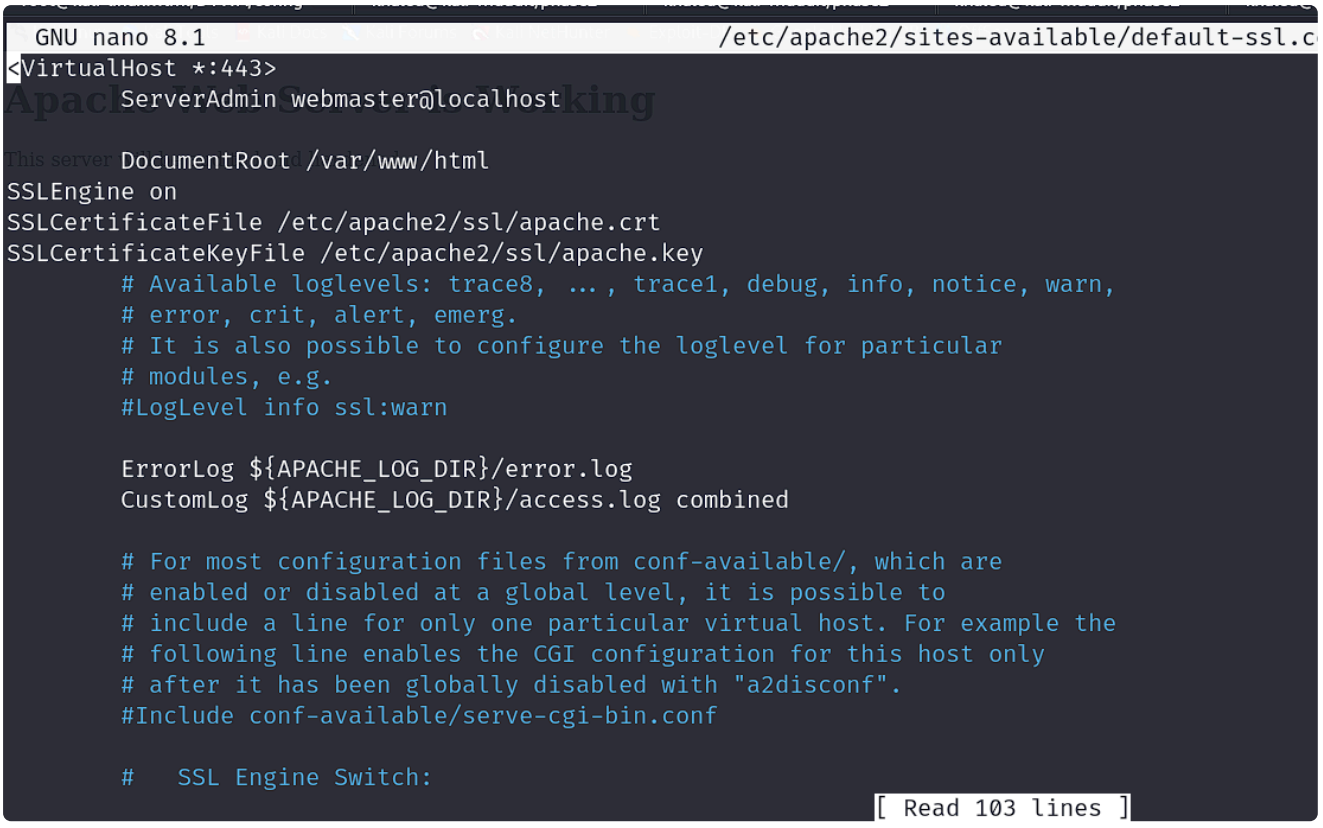
Vérification ou ajout des directives suivantes :

```
SSLEngine on
SSLCertificateFile /etc/apache2/ssl/apache.crt
SSLCertificateKeyFile /etc/apache2/ssl/apache.key
```

Redémarrage du service Apache :

```shell
sudo systemctl restart apache2
```

- **Extrait du fichier default-ssl.conf montrant les chemins du certificat et de la clé :**



---

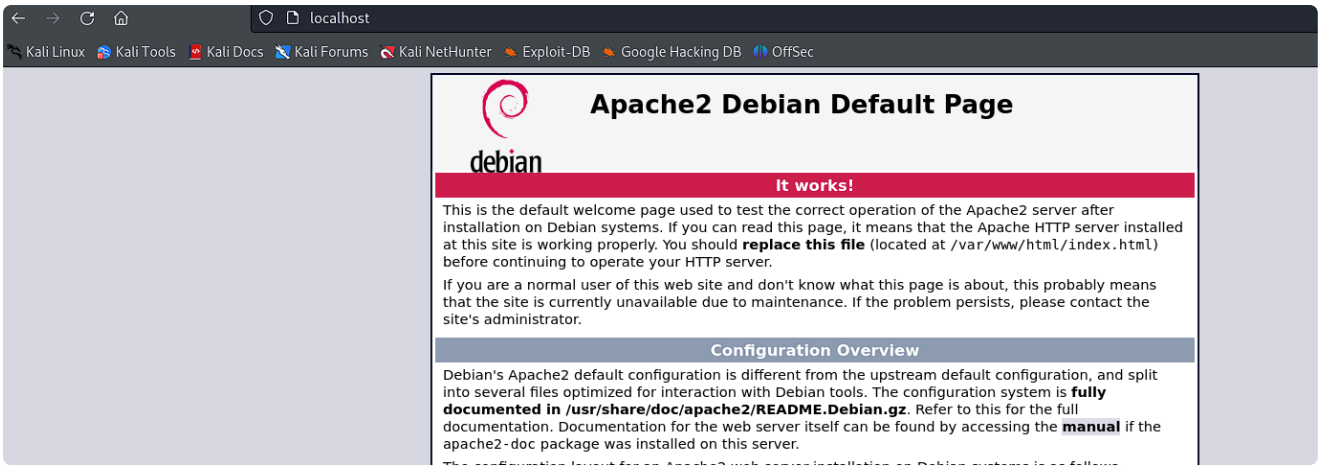
5. Vérification du fonctionnement HTTPS

Test via navigateur

Accès à l'URL suivante :

```
https://localhost
```

Un avertissement de sécurité apparaît, ce qui est normal dans le cas d'un certificat auto-signé.



Test via ligne de commande

```shell
curl -k -I https://localhost
```

La réponse attendue est :

```
HTTP/1.1 200 OK
```

- Terminal montrant le retour `HTTP/1.1 200 OK` via HTTPS

```
┌──(khaled㉿kali-linux-0)-[~/web-audit/phase2]
└─$ curl -k -I https://localhost
HTTP/1.1 200 OK
Date: Sun, 14 Dec 2025 19:43:27 GMT
Server: Apache
Last-Modified: Sun, 14 Dec 2025 17:33:39 GMT
ETag: "bf-645ece52180be"
Accept-Ranges: bytes
Content-Length: 191
Vary: Accept-Encoding
Content-Type: text/html
```

---

6. Désactivation des protocoles TLS obsolètes

Objectif

Éliminer l'utilisation de protocoles cryptographiques connus pour être vulnérables.

Configuration appliquée

Édition du fichier de sécurité Apache :

```shell
sudo nano /etc/apache2/conf-enabled/security.conf
```

Ajout ou modification de la directive suivante :

```
SSLProtocol all -SSLv2 -SSLv3 -TLSv1 -TLSv1.1
```

Redémarrage d'Apache :

```SHELL
sudo systemctl restart apache2
```

- **Fichier `security.conf` montrant la directive `SSLProtocol`**

```
GNU nano 8.1                    /etc/apache2/conf-enabled/security.conf
# Changing the following options will not really affect the security of the
# server, but might make attacks slightly more difficult in some cases.
SSLProtocol all -SSLv2 -SSLv3 -TLSv1 -TLSv1.1
ServerTokens Prod
ServerSignature Off
SSLCipherSuite HIGH:!aNULL:!MD5
SSLHonorCipherOrder on
# ServerTokens
# This directive configures what you return as the Server HTTP response
# Header. The default is 'Full' which sends information about the OS-Type
# and compiled in modules.
# Set to one of:  Full | OS | Minimal | Minor | Major | Prod
# where Full conveys the most information, and Prod the least.
#ServerTokens Minimal
#ServerTokens OS
#ServerTokens Full

#
# Optionally add a line containing the server version and virtual host
# name to server-generated pages (internal error documents, FTP directory
# listings, mod_status and mod_info output etc., but not CGI generated
# documents or custom error documents).
# Set to "EMail" to also include a mailto: link to the ServerAdmin.
# Set to one of:  On | Off | EMail
```

---

7. Amélioration des suites de chiffrement

Objectif

Forcer l'utilisation de suites de chiffrement robustes.

Configuration appliquée

Dans `security.conf` ou `default-ssl.conf` :

```
SSLCipherSuite HIGH:!aNULL:!MD5
SSLHonorCipherOrder on
```

Redémarrage du service :

```SHELL
sudo systemctl restart apache2
```

Ces paramètres garantissent l'utilisation de chiffrements forts et excluent les algorithmes faibles.

8. Vérifications finales

Vérification de l'écoute sur le port 443

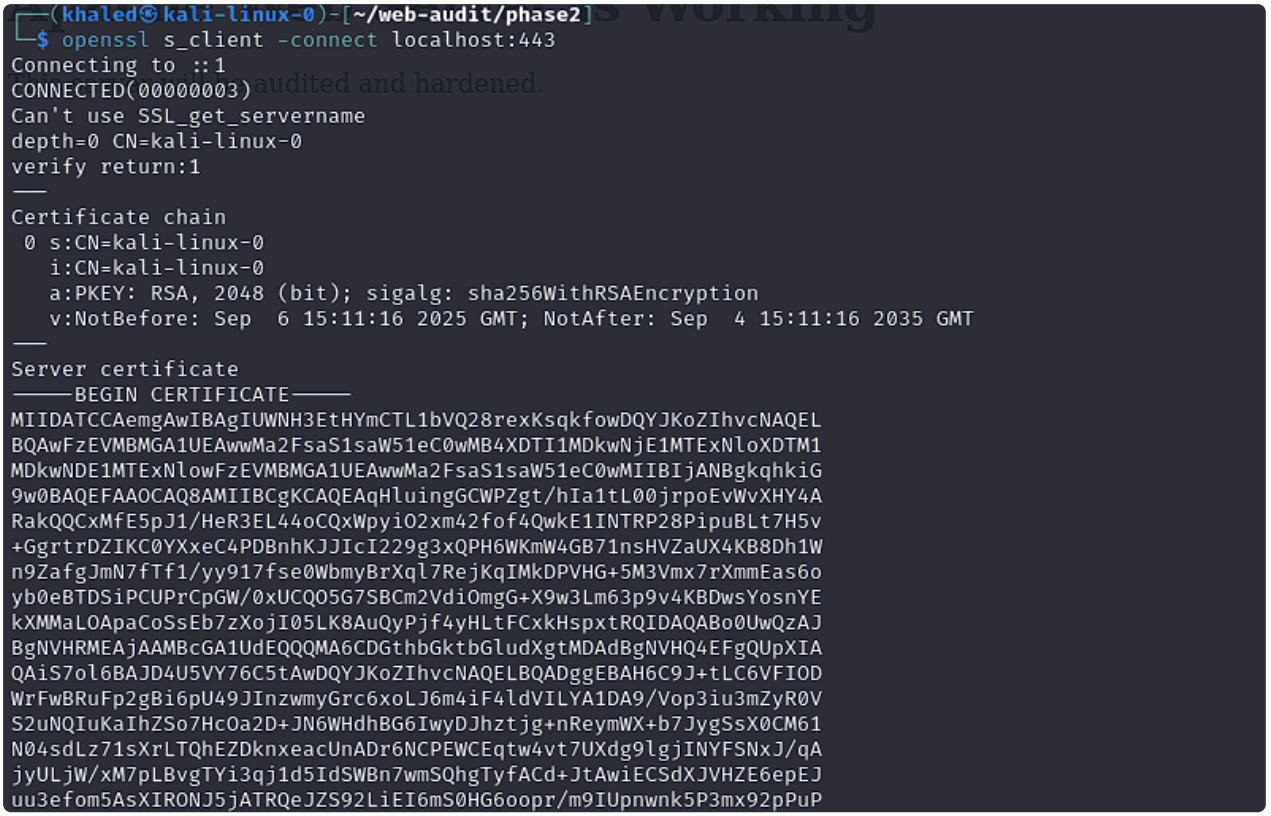
```shell
ss -tulpn | grep 443
```

Inspection du handshake TLS

```shell
openssl s_client -connect localhost:443
```

Cette commande permet de confirmer :

- L'utilisation de TLS
- La validité du certificat
- Les suites de chiffrement actives
- Sortie de `openssl s_client` montrant une connexion TLS établie



---

9. Bilan de la phase 6

À l'issue de cette phase, les communications du serveur web sont sécurisées :

- HTTPS activé et fonctionnel
- Certificat TLS auto-signé configuré
- Protocoles obsolètes désactivés
- Suites de chiffrement renforcées

Ce niveau de configuration est **parfaitement adapté à un projet académique de niveau intermédiaire** et démontre une compréhension claire des mécanismes de sécurisation des communications web.

---

10. Conclusion

Le serveur Apache est désormais capable de fournir des contenus via un canal chiffré, réduisant significativement les risques d'interception ou de modification des données échangées.
Cette étape complète le durcissement du serveur web et prépare la phase finale du projet, dédiée à la **réévaluation de la sécurité après durcissement**.

Phase 7 : Audit post-durcissement et vérification finale

(Post-Hardening Audit)

---

1. Introduction de la phase

La dernière phase du projet **Audit et durcissement d'un serveur web** consiste à réaliser un **audit de vérification après durcissement**.
L'objectif est de confirmer que les mesures appliquées lors des phases précédentes ont **réellement amélioré la posture de sécurité**, sans dégrader la disponibilité ni le fonctionnement du service.

Cette phase permet de fermer la boucle méthodologique suivante :

> Audit initial → Analyse des risques → Durcissement → Vérification

Elle apporte une **preuve concrète** que le durcissement n'est pas uniquement théorique.

---

2. Objectifs de la vérification post-durcissement

Les objectifs principaux de cette phase sont :

- Vérifier que les faiblesses identifiées lors de la Phase 2 ont été corrigées ou réduites
- Confirmer l'efficacité des configurations de sécurité mises en place
- S'assurer que les services web restent accessibles et fonctionnels
- Comparer objectivement l'état du serveur **avant** et **après** durcissement

---

3. Vérification de l'état des services

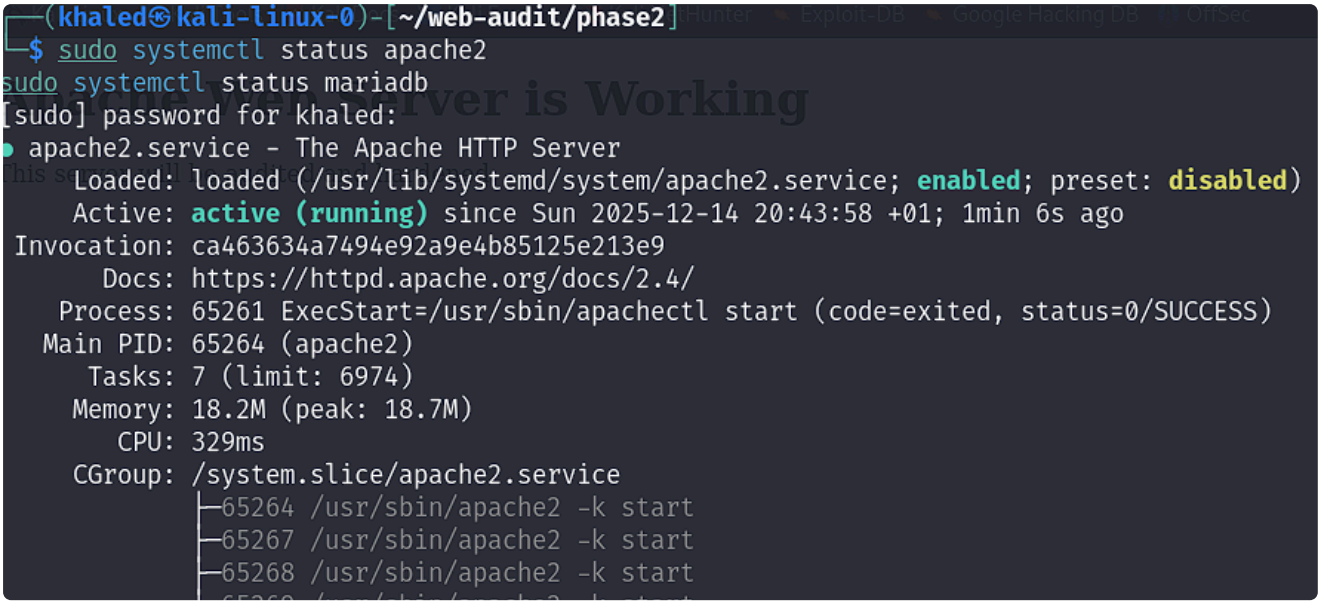
Services critiques

Les services essentiels au projet sont vérifiés :

```shell
sudo systemctl status apache2
sudo systemctl status mariadb
```

Résultat attendu

- Les services sont actifs (*active (running)*)
- Aucune erreur critique n'est signalée
- **État du service Apache montrant *active (running)* && (Optionnel) état de MariaDB s'il est utilisé**



4. Réanalyse des ports ouverts (Nmap)

Scan des ports après durcissement

```shell
sudo nmap -p- -sS --open localhost
```

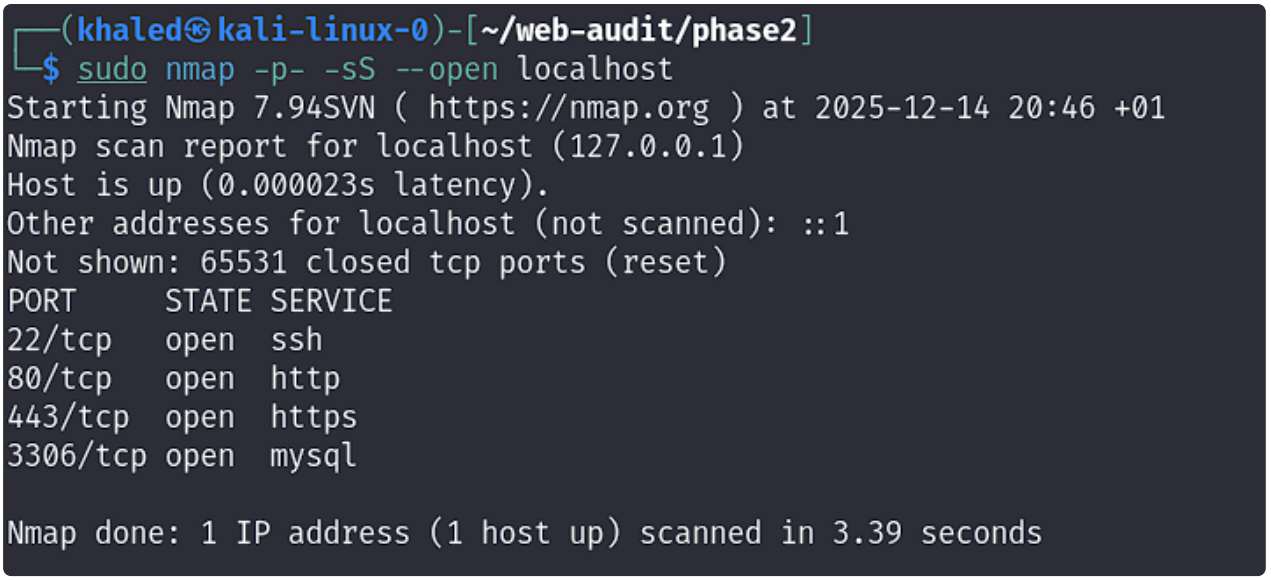
Analyse des résultats

Après durcissement :

- Les ports ouverts sont limités
- En général, seuls **80 (HTTP)** et **443 (HTTPS)** sont visibles
- Aucun service inutile n'est exposé

Cette réduction confirme l'efficacité du durcissement système et du pare-feu.

- Résultat Nmap montrant uniquement les ports nécessaires



---

5. Réanalyse ciblée des services web

Scan HTTP/HTTPS avec scripts Nmap

```shell
sudo nmap -p 80,443 -sV -sC --script \
http-methods,http-headers,http-security-headers,http-enum,http-trace \
localhost
```

Points vérifiés

- Méthode HTTP TRACE désactivée
- Accès à `/server-status` non autorisé
- Amélioration ou apparition de certains en-têtes de sécurité
- HTTPS actif sur le port 443

- Résultat Nmap montrant TRACE désactivé et HTTPS actif

```
┌──(khaled㉿kali-linux-0)-[~/web-audit/phase2]
└─$ sudo nmap -p 80,443 -sV -sC --script \
http-methods,http-headers,http-security-headers,http-enum,http-trace \
localhost
Starting Nmap 7.94SVN ( https://nmap.org ) at 2025-12-14 20:46 +01
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00013s latency).
Other addresses for localhost (not scanned): ::1

Bug in http-security-headers: no string output.
PORT    STATE SERVICE  VERSION
80/tcp  open  http     Apache httpd
| http-methods:
|_  Supported Methods: HEAD GET POST OPTIONS
| http-headers:
|   Date: Sun, 14 Dec 2025 19:46:38 GMT
|   Server: Apache
|   Last-Modified: Sun, 14 Dec 2025 17:33:39 GMT
|   ETag: "bf-645ece52180be"
|   Accept-Ranges: bytes
|   Content-Length: 191
|   Vary: Accept-Encoding
|   Connection: close
|   Content-Type: text/html
|
|_  (Request type: HEAD)
|_http-server-header: Apache
443/tcp open  ssl/http Apache httpd
| http-methods:
|_  Supported Methods: HEAD GET POST OPTIONS
| http-security-headers:
|   Strict_Transport_Security:
|_    HSTS not configured in HTTPS Server
| http-headers:
|   Date: Sun, 14 Dec 2025 19:46:38 GMT
|   Server: Apache
|   Last-Modified: Sun, 14 Dec 2025 17:33:39 GMT
|   ETag: "bf-645ece52180be"
```

---

6. Vérification de la divulgation d'informations Apache

Tests via curl

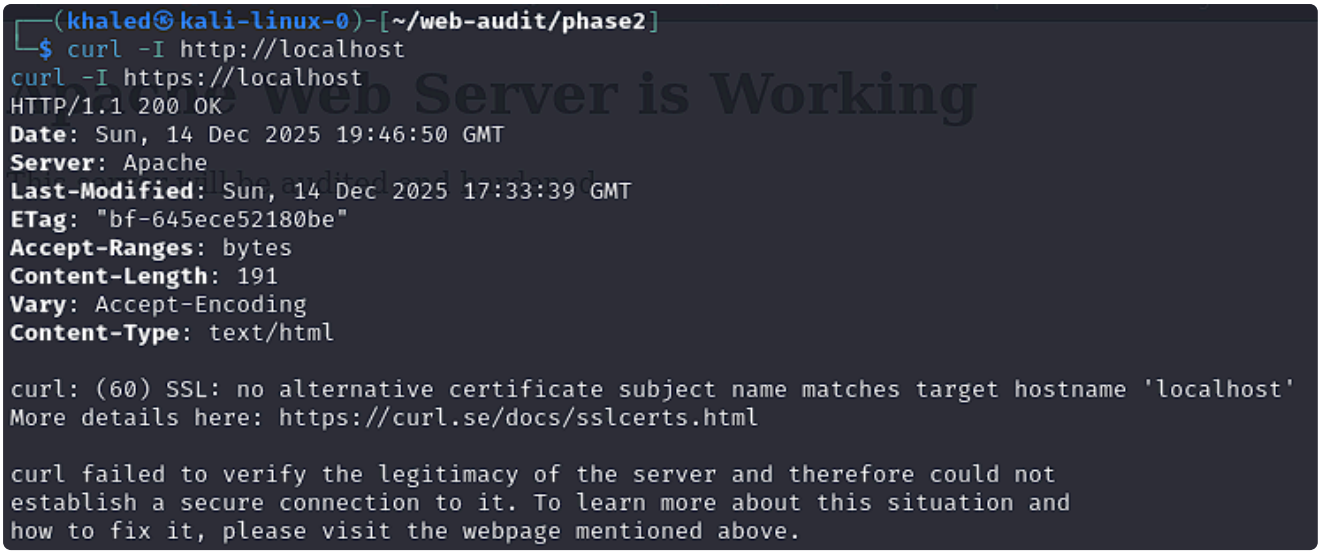
```shell
curl -I http://localhost
curl -I https://localhost
```

Résultat attendu

- Le header `Server` est présent
- Aucune information sensible n'est divulguée :
  - Pas de version détaillée
  - Pas de système d'exploitation
  - Pas de modules ou version PHP

Dans un contexte académique, l'affichage de `Server: Apache` seul est acceptable.

- En-têtes HTTP/HTTPS montrant une divulgation minimale



---

7. Réexécution du scan Nikto

Scan post-durcissement

```shell
nikto -h http://localhost
```

Comparaison avec la Phase 2

Les résultats montrent généralement :

- Réduction du nombre d'alertes
- Disparition de :
  - HTTP TRACE
  - `/server-status`
- Certaines alertes mineures peuvent subsister (headers manquants), ce qui reste acceptable académiquement
- Extrait du scan Nikto post-durcissement montrant moins d'alertes



---

8. Vérification de HTTPS et de TLS

Test HTTPS

```shell
curl -k -I https://localhost
```

Résultat attendu :

- `HTTP/1.1 200 OK`

Vérification des protocoles TLS

```shell
openssl s_client -connect localhost:443 -tls1
```

```shell
openssl s_client -connect localhost:443 -tls1_1
```

Résultat attendu

- Les connexions utilisant TLS 1.0 et 1.1 sont refusées
- Les protocoles modernes (TLS 1.2 / 1.3) sont utilisés
- **Refus de connexion avec TLS obsolète && Connexion réussie avec TLS moderne**

```
┌──(khaled㉿kali-linux-0)-[~/web-audit/phase2]
└─$ openssl s_client -connect localhost:443 -tls1
Connecting to ::1
CONNECTED(00000003)
4067C2216B7F0000:error:0A00042E:SSL routines:ssl3_read_bytes:tlsv1 alert protocol version:../ssl/record/rec_layer_s3.c:916:SSL alert number 70
---
no peer certificate available
---
No client certificate CA names sent
---
SSL handshake has read 7 bytes and written 140 bytes
Verification: OK
---
New, (NONE), Cipher is (NONE)
Protocol: TLSv1
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol  : TLSv1
    Cipher    : 0000
    Session-ID:
    Session-ID-ctx:
    Master-Key:
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    Start Time: 1765741718
    Timeout   : 7200 (sec)
    Verify return code: 0 (ok)
    Extended master secret: no
```

```
┌──(khaled㉿kali-linux-0)-[~/web-audit/phase2]
└─$ openssl s_client -connect localhost:443 -tls1_1
Connecting to ::1
CONNECTED(00000003)
40F7330CCA7F0000:error:0A00042E:SSL routines:ssl3_read_bytes:tlsv1 alert protocol version:../ssl/record/rec_layer_s3.c:916:SSL alert number 70
---
no peer certificate available
---
No client certificate CA names sent
---
SSL handshake has read 7 bytes and written 143 bytes
Verification: OK
---
New, (NONE), Cipher is (NONE)
Protocol: TLSv1.1
Secure Renegotiation IS NOT supported
Compression: NONE
Expansion: NONE
No ALPN negotiated
SSL-Session:
    Protocol  : TLSv1.1
    Cipher    : 0000
    Session-ID:
    Session-ID-ctx:
    Master-Key:
    PSK identity: None
    PSK identity hint: None
    SRP username: None
    Start Time: 1765741721
    Timeout   : 7200 (sec)
    Verify return code: 0 (ok)
    Extended master secret: no
```

---

9. Vérification de la disponibilité du service

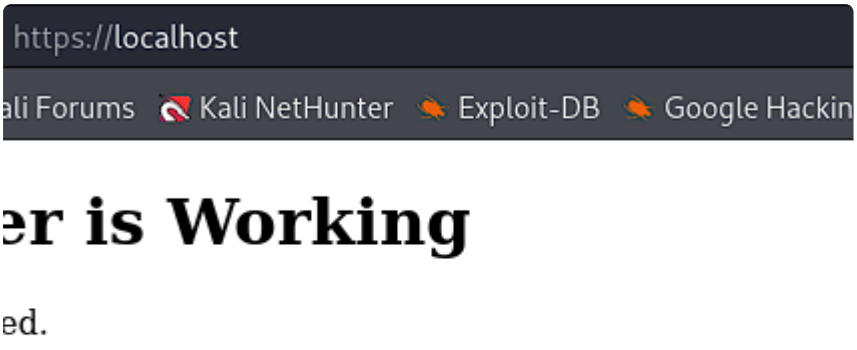
Test fonctionnel

Accès manuel via navigateur :

- http://localhost
- https://localhost

Résultat attendu

- Le site est accessible en HTTP et HTTPS
- Aucun dysfonctionnement visible
- Le durcissement n'a pas impacté la disponibilité du service
- **Page web affichée correctement en HTTPS**



---

10. Bilan de la phase 7

La réévaluation post-durcissement montre que :

- Le nombre de faiblesses détectées a diminué
- Les configurations de sécurité sont effectives
- Les informations sensibles sont mieux protégées
- Le serveur web reste pleinement fonctionnel

Les mesures de durcissement appliquées ont donc **amélioré concrètement la sécurité**, tout en respectant les contraintes de disponibilité.

---

11. Conclusion finale

Cette phase confirme que le projet de **durcissement du serveur web** a atteint ses objectifs.
La comparaison entre l'état initial et l'état final démontre une **progression claire et mesurable** du niveau de sécurité.

Le serveur est désormais mieux protégé contre :

- L'exposition inutile de services
- La divulgation d'informations sensibles
- Les communications non chiffrées

Ce travail constitue un **projet académique complet, cohérent et techniquement justifié**.
